KPR INSTITUTE OF ENGINEERING AND TECHNOLOGY

Department of Artificial Intelligence and Data Science

Academic Year 2026 – 2027 (Odd Semester)

U21ADP05 – EXPLORATORY DATA ANALYSIS AND VISUALIZATION

ASSIGNMENT II – CASE STUDY PROJECT REPORT

Exploratory Data Analysis and Visualization of a Real-World Dataset:

Breast Cancer Wisconsin (Diagnostic)

with a Deep Neural Network (MLP) for Benign / Malignant Classification

Student Name	Your Full Name
Roll No.	Your Roll Number
Department	Artificial Intelligence and Data Science
Year / Semester	III Year / Semester V (Batch 2023 – 2027)
Course Coordinator	Mr. Rushikesh Kadam
Date of Submission	15/10/2026
GitHub Repository	https://github.com/your-username/breast-cancer-eda-visualization

Abstract

Early and accurate discrimination between benign and malignant breast masses is an important clinical need. This project applies exploratory data analysis, statistical analysis and visualization to the Breast Cancer Wisconsin (Diagnostic) dataset, which describes 569 patients using 30 nuclear features. Following the seven stages of visualization, the data were cleaned, skewed features were power-transformed and standardised, and the records were split 70/15/15. Seventeen visualizations, distribution fitting and hypothesis tests showed that size and concavity features separate the classes strongly and that many features are highly collinear. A multi-layer perceptron (30-64-32-16-1), tuned over 24 configurations, achieved 98.8% test accuracy, 96.9% recall and near-perfect AUC, and 96.5% five-fold cross-validated accuracy, comparable to logistic-regression and random-forest baselines.

Keywords: exploratory data analysis, data visualization, breast cancer, statistical distributions, multi-layer perceptron, ROC–AUC.

1. Introduction and Objective

1.1 Background

Fine-needle aspiration (FNA) is a minimally invasive procedure in which a small sample of cells is drawn from a breast mass and examined under a microscope. By digitising the microscope image and measuring the size and shape of the cell nuclei, a pathologist's visual judgement can be complemented with objective numerical features (Street et al., 1993). The resulting tabular data are an ideal real-world case for exploratory data analysis (EDA): the variables are numeric, strongly inter-related, unevenly distributed and linked to a clear binary outcome (benign or malignant).

1.2 Problem statement

Given 30 numeric descriptors of cell nuclei for 569 patients, (i) understand how the descriptors are distributed and related to each other and to the diagnosis, (ii) communicate these findings through effective visualizations, and (iii) build and evaluate a deep neural network that predicts whether a tumour is malignant.

1.3 Objectives

- Select a real-world dataset with at least 15 features and load, clean and understand it (missing values, duplicates, outliers, transformation, encoding, scaling, train/validation/test split).
- Apply the seven stages of visualization (Acquire, Parse, Filter, Mine, Represent, Refine, Interact) and create more than five meaningful visualizations, each explained by what it shows, why it was created and the insight obtained.
- Perform univariate, bivariate and multivariate statistical analysis, including continuous and discrete probability-distribution analysis.
- Build, tune and evaluate at least one deep-learning model (multi-layer perceptron) and visualise its results (loss/accuracy curves, confusion matrix, ROC–AUC, error distribution, feature importance).
- Document the project and publish the code, data, report and README in a structured public GitHub repository.

1.4 Literature review

Dataset and feature extraction. Street, Wolberg and Mangasarian (1993) described a system that computes nuclear features such as radius, texture, perimeter, area, smoothness, compactness, concavity, symmetry and fractal dimension from digitised FNA images and uses them to separate benign from malignant samples. The data are distributed through the UCI Machine Learning Repository (Dua & Graff, 2019) and have become a standard benchmark for classification and visualization exercises.

Exploratory data analysis and visualization. Tukey (1977) established EDA as the practice of looking at data through graphical and numerical summaries before modelling; the box plot and the 1.5×IQR outlier rule used in this report originate from that work. Fry (2007) organised the visualization process into seven stages – acquire, parse, filter, mine, represent, refine and

interact – which structure the workflow of this project. Plots are produced with Matplotlib (Hunter, 2007) and Seaborn (Waskom, 2021).

Neural networks. Goodfellow, Bengio and Courville (2016) describe multi-layer feed-forward networks (multi-layer perceptrons) trained by back-propagation, together with the regularisation techniques used here – L2 weight penalties and early stopping. Networks are optimised with Adam (Kingma & Ba, 2015) and implemented with scikit-learn (Pedregosa et al., 2011).

1.5 Methodology overview

Table 1: *Project workflow mapped to the seven stages of visualization*

Stage	Activity in this project	Report section
1 Acquire	Load the UCI WDBC data (569 × 31 CSV)	2
2 Parse	Data types, shape, feature groups, summary statistics	2
3 Filter	Missing values, duplicates, IQR outliers, zero-variance check	3, 4.2
4 Mine	Skewness, hypothesis tests, distribution fitting, correlation, PCA	3.3, 4.3
5 Represent	17 static visualizations (distribution, relationship, model-evaluation)	4, 5, 6
6 Refine	Power transform, scaling, stratified split, consistent colour-blind-safe design	3.1, 4.4
7 Interact	Self-contained interactive HTML explorer (any two features, class filter, hover)	4.5

2. Dataset Description

2.1 Source and collection

The **Breast Cancer Wisconsin (Diagnostic)** dataset (WDBC) was created by Dr. William H. Wolberg, W. Nick Street and Olvi L. Mangasarian at the University of Wisconsin and is published in the UCI Machine Learning Repository (https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic). Each record corresponds to one digitised image of an FNA of a breast mass; ten real-valued characteristics are computed for every cell nucleus in the image and then summarised per image. A copy of the same data is bundled with scikit-learn (`load_breast_cancer`), and the CSV used in this project is stored in the repository under `data/`.

2.2 Size and fields

Table 2: *Dataset overview*

Property	Value
Records (patients / images)	569
Input features	30 (all numeric, continuous)
Target	`diagnosis` – M (malignant) / B (benign)
Class distribution	357 benign (62.7%) and 212 malignant (37.3%)
Missing values / duplicate rows	0 / 0
Task type	Binary classification; supervised

Table 3: *Feature groups (10 measurements × 3 statistics = 30 features)*

Measurement	Meaning	Columns
radius	mean distance from centre to perimeter points	mean_radius, radius_error, worst_radius
texture	standard deviation of grey-scale values	mean_texture, texture_error, worst_texture
perimeter	nucleus perimeter	mean_perimeter, perimeter_error, worst_perimeter
area	nucleus area	mean_area, area_error, worst_area
smoothness	local variation in radius lengths	mean_/_error/worst_ smoothness
compactness	perimeter² / area − 1.0	mean_/_error/worst_ compactness
concavity	severity of concave portions of the contour	mean_/_error/worst_ concavity
concave points	number of concave portions of the contour	mean_/_error/worst_ concave_points
symmetry	symmetry of the nucleus	mean_/_error/worst_ symmetry
fractal dimension	“coastline approximation” − 1	mean_/_error/worst_ fractal_dimension

“error” denotes the standard error of the measurement over all nuclei in the image and “worst” the mean of the three largest values. A full data dictionary is provided in `data/DATA_DICTIONARY.md`.

2.3 Basic statistics

Table 4: *Descriptive statistics of selected features (all 569 patients)*

Feature	Mean	Std. dev.	Min	Median	Max
mean radius	14.13	3.52	6.98	13.37	28.11
mean texture	19.29	4.30	9.71	18.84	39.28
mean area	654.89	351.91	143.50	551.10	2501.00
mean smoothness	0.096	0.014	0.053	0.096	0.163
mean concave points	0.049	0.039	0.000	0.034	0.201
area error	40.34	45.49	6.80	24.53	542.20
worst perimeter	107.26	33.60	50.41	97.66	251.20
worst area	880.58	569.36	185.20	686.50	4254.00

The scales differ by several orders of magnitude (for example `mean_smoothness` ≈ 0.1 versus `worst_area` ≈ 880), which already signals that standardisation is required before neural-network training. The class ratio is 1.68 : 1 (benign : malignant) – moderately imbalanced – so a classifier that always predicts “benign” would already score 62.7% accuracy; accuracy alone is therefore not a sufficient metric.

3. EDA and Preprocessing

3.1 Methods used

Table 5: *Data-cleaning and preprocessing steps and their outcome*

Step	Method	Outcome
Missing values	`isna().sum()`; generic median imputation kept in pipeline	0 missing values – none needed
Duplicates	`duplicated()`	0 duplicate rows
Zero-variance features	unique-value count per column	none
Outlier detection	Tukey rule: value < Q1 − 1.5·IQR or > Q3 + 1.5·IQR	171 rows (30.1%) contain ≥ 1 outlier; **retained** (see 3.2)
Encoding	`diagnosis`: M → 1 (positive class), B → 0	binary numeric target
Splitting	stratified 70 / 15 / 15 (random_state = 42), done **before** any fitting	train 398 / validation 85 / test 86; malignant 148 / 32 / 32
Transformation	Yeo-Johnson power transform on the 22 features with |skewness| > 1, fitted on **train only**	mean |skewness| 1.74 → 0.25; no feature remains with |skew| > 1
Normalisation / standardisation	z-score `StandardScaler`, fitted on **train only**	every feature has mean 0 and unit variance on the training set

3.2 Insights from the exploratory analysis

- **The data are clean.** There are no missing values or duplicates, so the effort concentrates on distribution shape, outliers and redundancy.
- **Outliers are biological, not errors.** 171 of 569 rows (30.1%) contain at least one IQR outlier, and 66.7% of those rows are malignant although malignant cases are only 37.3% of the data. Very large nuclei are exactly what malignancy produces, so removing them would delete the signal. They were kept and handled by the power transform and scaling instead.
- **Heavy skewness.** 22 of 30 features have |skewness| > 1; the most skewed are `area_error` (5.45), `concavity_error` (5.11) and `fractal_dimension_error` (3.92). A first attempt with `log1p` barely changed features whose values are much smaller than 1 (`concavity_error` 5.11 → 4.23; `fractal_dimension_error` 3.92 → 3.87), so the Yeo-Johnson transform was used instead (`concavity_error` → 0.14, `fractal_dimension_error` → 0.24).
- **Strong redundancy.** 21 of the 435 feature pairs have |r| > 0.9; `mean_radius` and `mean_perimeter` correlate at r = 0.998. Seven principal components retain 90% of the variance, so the 30 features carry far fewer independent signals.

3.3 Statistical findings: benign versus malignant

Because most features are not normally distributed (Section 4.3), the two classes were compared with the non-parametric **Mann–Whitney U test**, and the practical size of each difference was measured by **Cohen's d**. 27 of 30 features differ at α = 0.05 and 26 remain significant after a Bonferroni correction for 30 tests (α = 0.0017); nine features have |d| > 2, which is a very large effect. Only `smoothness_error` (p = 0.21),

`mean_fractal_dimension` ($p = 0.54$) and `texture_error` ($p = 0.64$) show no difference. Table 6 lists the eight strongest discriminators – all describe nuclear size or concave shape.

Table 6: *Top-8 features by effect size (Mann–Whitney U p-values; M = malignant, B = benign)*

Feature	Mean (M)	Mean (B)	Cohen's d	p-value
worst concave points	0.182	0.074	2.69	< 0.001
worst perimeter	141.4	87.0	2.60	< 0.001
mean concave points	0.088	0.026	2.54	< 0.001
worst radius	21.1	13.4	2.54	< 0.001
mean perimeter	115.4	78.1	2.29	< 0.001
worst area	1422.3	558.9	2.23	< 0.001
mean radius	17.5	12.1	2.21	< 0.001
mean area	978.4	462.8	2.08	< 0.001

4. Data Visualization

This section presents eleven visualizations (the minimum required is five). They follow the seven stages of visualization: stages 1–4 produce the facts, stage 5 (Represent) is Sections 4.1–4.4, stage 6 (Refine) is the transformation and design clean-up shown in Section 4.4, and stage 7 (Interact) is Section 4.5. All plots use the same colour-blind-safe palette (blue = benign, vermillion = malignant), titled axes and legends.

4.1 Univariate analysis and distribution exploration

Visual 1 – Class distribution (bar chart and pie chart)

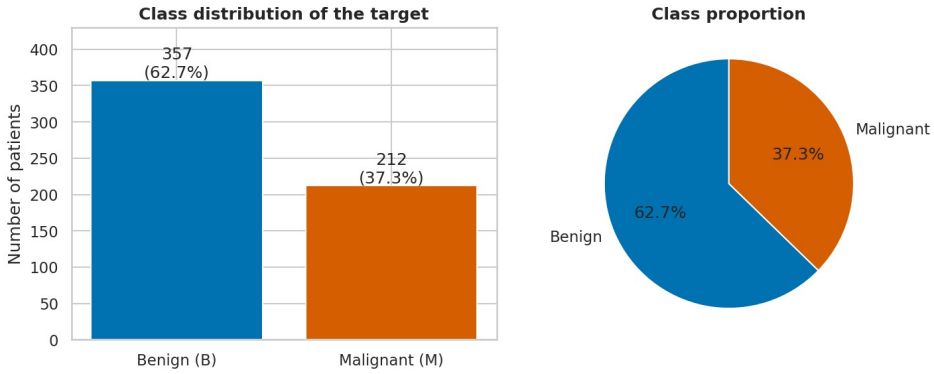


Figure 1: *Class distribution of the diagnosis target.*

What it shows: the number and share of benign and malignant patients.

Why it was created: the target's balance decides which evaluation metrics are meaningful and whether stratified splitting is needed.

Insights obtained: 357 patients (62.7%) are benign and 212 (37.3%) malignant – a moderate 1.68 : 1 imbalance. It is mild enough that no resampling is required, but accuracy alone would be misleading (a majority-class guess scores 62.7%), so recall, precision, F1 and AUC are reported and all splits are stratified.

Visual 2 – Histograms with kernel-density curves by diagnosis

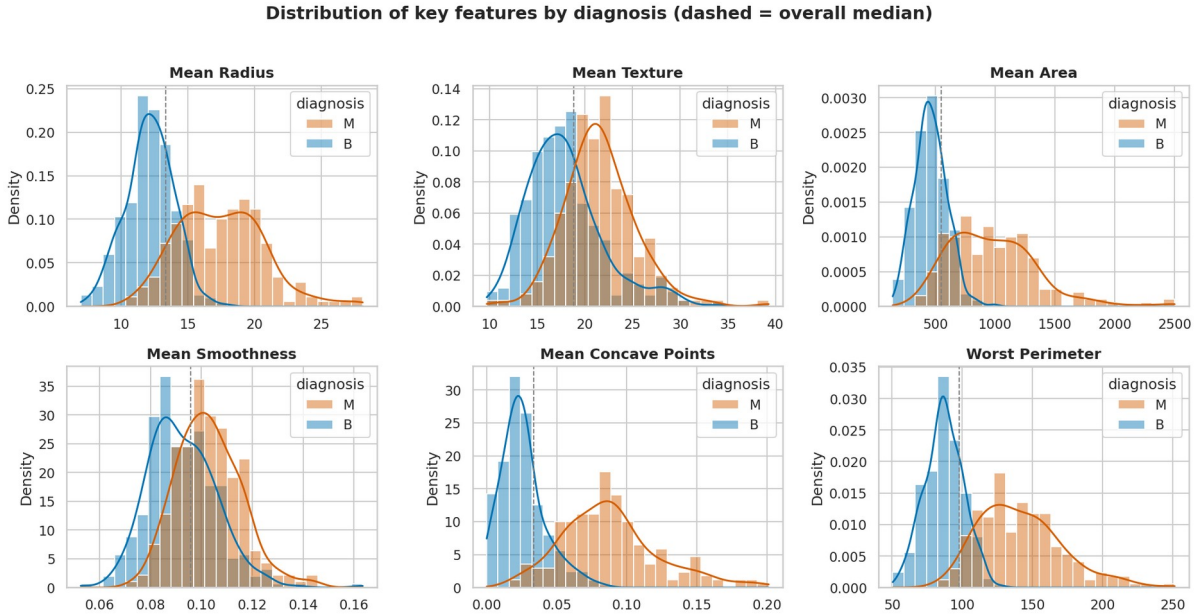


Figure 2: *Distribution of six key features for benign and malignant tumours (density-normalised; dashed line = overall median).*

What it shows: the distribution of six representative features, drawn separately for each class.

Why it was created: to inspect shape (symmetry, skewness, modes) and to see how much the two classes overlap, feature by feature.

Insights obtained: Size and shape features (mean radius, mean area, worst perimeter, mean concave points) are shifted to the right and are wider for malignant tumours, whereas benign tumours form a narrow peak. Mean concave points separates the classes almost completely (class medians 0.023 versus 0.086). Mean texture and mean smoothness overlap heavily and are weak discriminators on their own. Area is clearly right-skewed, motivating the power transform.

Visual 3 – Box plots by diagnosis

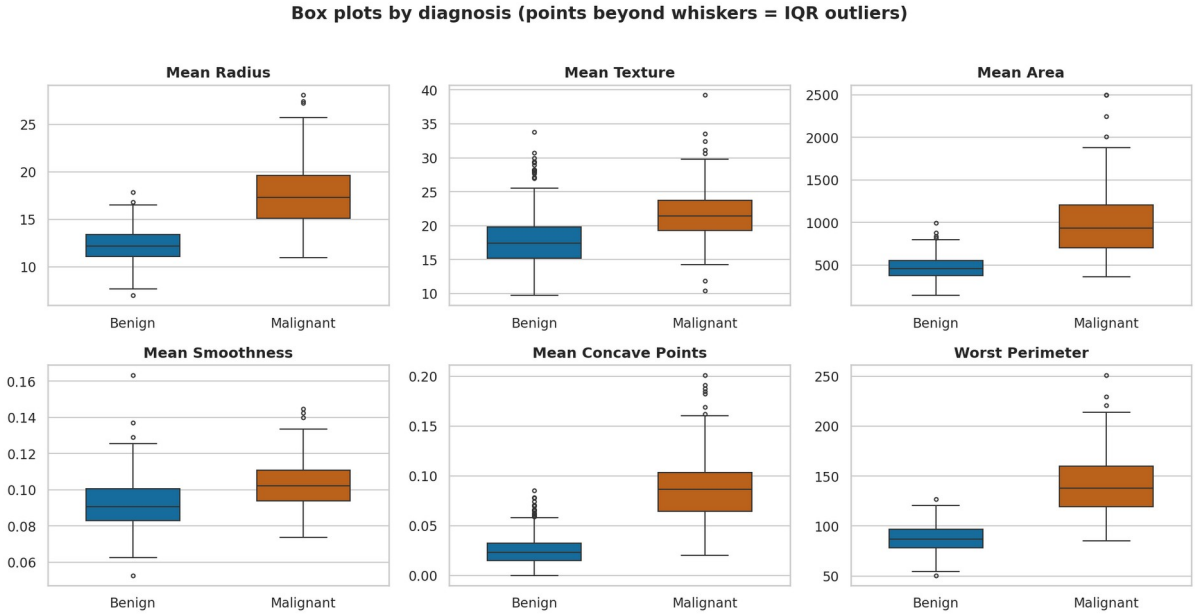


Figure 3: *Box plots of the same six features by diagnosis.*

What it shows: the median, inter-quartile range and outliers of each feature per class.

Why it was created: box plots give a compact, robust comparison of centre and spread and show outliers explicitly.

Insights obtained: For all six features the malignant median lies above the benign upper quartile (for example mean area 932 versus a benign Q3 of 551; mean concave points 0.086 versus 0.033). For the size and concavity features the malignant boxes are also taller, i.e. malignant tumours vary more. Benign tumours show many high-side outliers for mean texture (18) and mean concave points (17), meaning a few benign tumours look unusual, which explains the overlap region later seen in the scatter plots.

Visual 4 – Outlier prevalence per feature

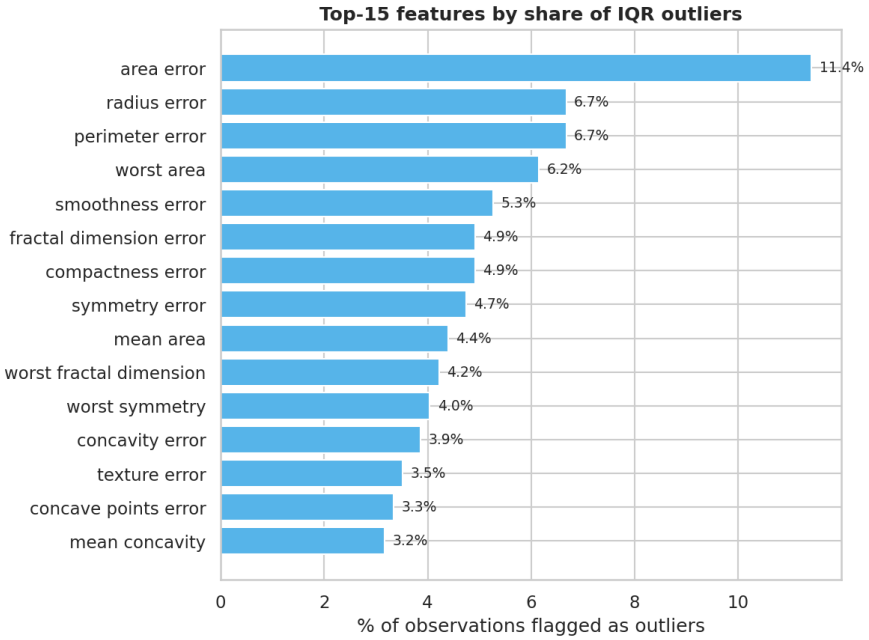


Figure 4: *Top-15 features by percentage of IQR outliers.*

What it shows: the share of observations flagged by the 1.5×IQR rule for the 15 most affected features.

Why it was created: to decide whether outlier handling is a minor or a major issue and which features drive it.

Insights obtained: `area_error` is the worst case (11.42% of rows), followed by `radius_error` and `perimeter_error` (6.68%) and `worst_area` (6.15%). Standard-error features dominate the list because they are strongly right-skewed. Since flagged rows are disproportionately malignant (66.7%), the outliers carry information and were retained.

4.2 Bivariate and multivariate analysis

Visual 5 – Correlation heat-map and correlation with the target

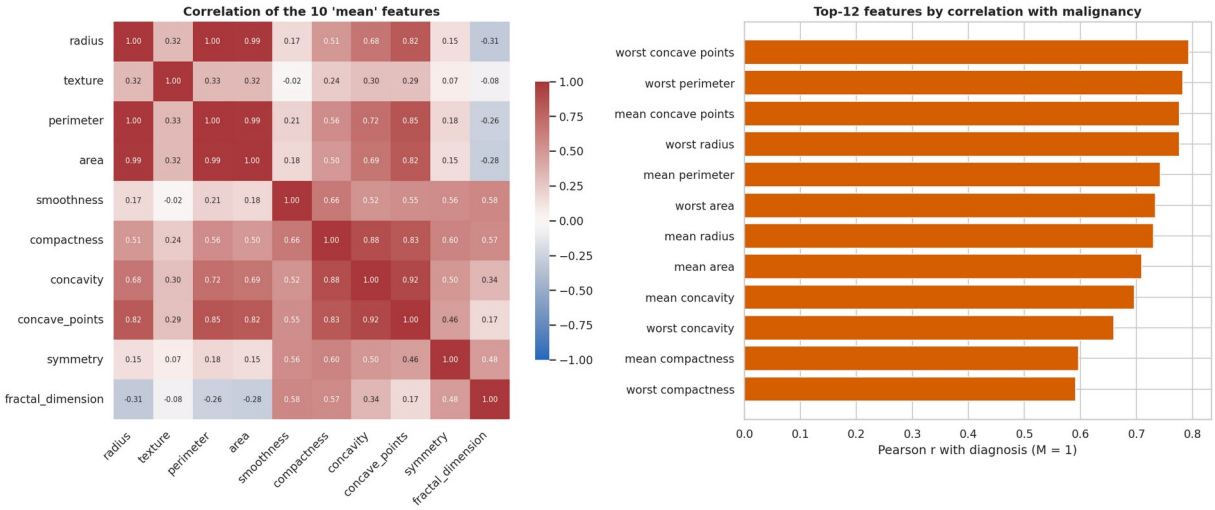


Figure 5: *Left: correlation matrix of the ten mean features. Right: twelve features most correlated with malignancy.*

What it shows: pairwise Pearson correlations among the ten "mean" features (left) and the features most correlated with the diagnosis (right).

Why it was created: to find redundant variables (multicollinearity) and rank features by linear association with the target.

Insights obtained: Radius, perimeter and area are almost perfectly correlated (r = 0.998 for radius–perimeter, 0.99 for radius–area), because perimeter and area are geometric functions of size. Concavity and concave points correlate at 0.92 and compactness and concavity at 0.88. Texture is only weakly related to the others (r ≈ 0.3), so it supplies independent information. The strongest target correlations are worst concave points (0.794), worst perimeter (0.783) and mean concave points (0.777); 21 of 435 feature pairs exceed |r| = 0.9, so models must tolerate collinearity.

Visual 6 – Scatter plots of selected feature pairs

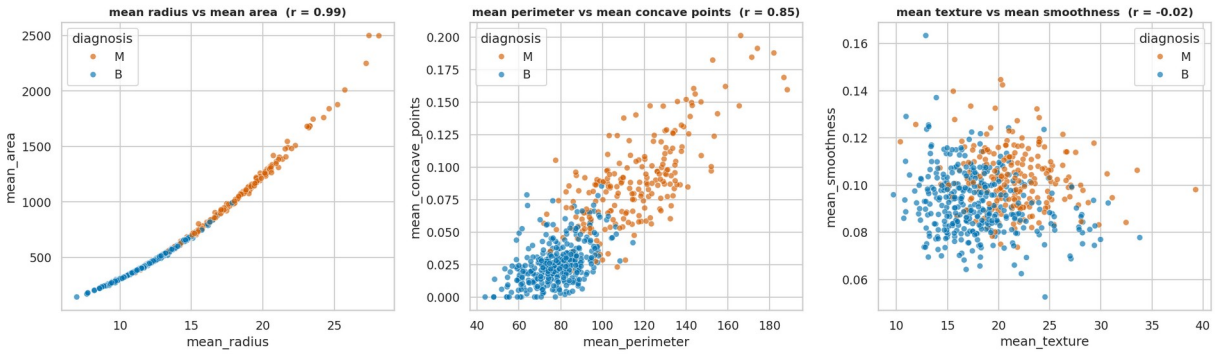


Figure 6: *Scatter plots coloured by diagnosis with Pearson r in each title.*

What it shows: three feature pairs representing a near-deterministic, a moderate and a null relationship.

Why it was created: scatter plots reveal the form of a relationship and class clusters that a single correlation number hides.

Insights obtained: Radius versus area is an almost noise-free curve (r = 0.987; area grows with the square of radius). Perimeter versus concave points (r = 0.851) shows malignant cases in the upper-right and benign in the lower-left with a narrow overlap band – a combination of size and shape separates the classes better than either alone. Texture versus smoothness is a shapeless cloud (r = -0.023) with strongly overlapping classes.

Visual 7 – Pair plot of five discriminative features

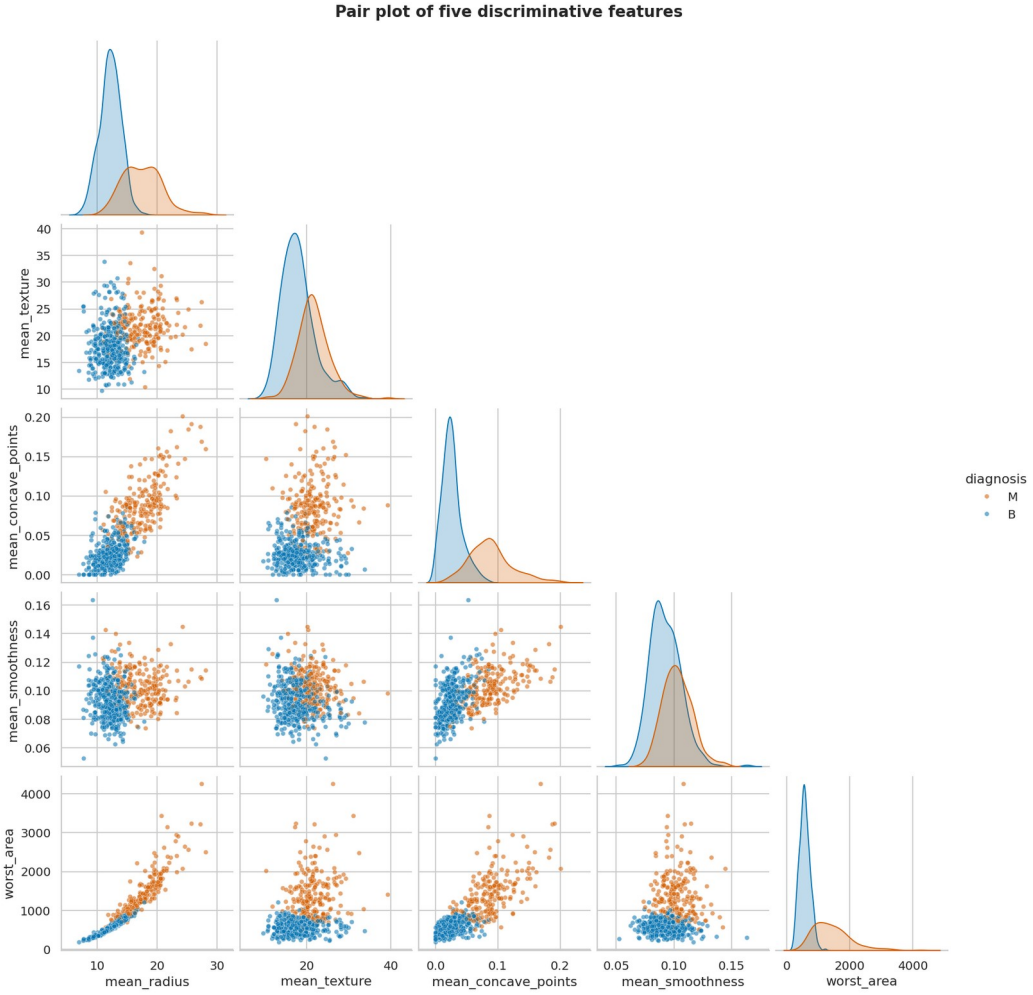


Figure 7: *Pair plot (lower triangle scatter, diagonal KDE) of five features.*

What it shows: all pairwise scatter plots and marginal densities of mean radius, mean texture, mean concave points, mean smoothness and worst area.

Why it was created: to look at several variables simultaneously and judge whether the classes are separable in combinations of features.

Insights obtained: Panels that combine a size feature with a concavity feature (e.g. worst area × mean concave points) display two nearly separated clouds, which can be divided by an almost straight line. This explains why even logistic regression performs strongly on this dataset and suggests that a small neural network will not need much depth. Panels that include texture or smoothness show heavy overlap.

4.3 Probability distributions: continuous and discrete

Visual 8 – Distribution fitting: Normal versus Log-normal, with Q–Q plots (continuous)

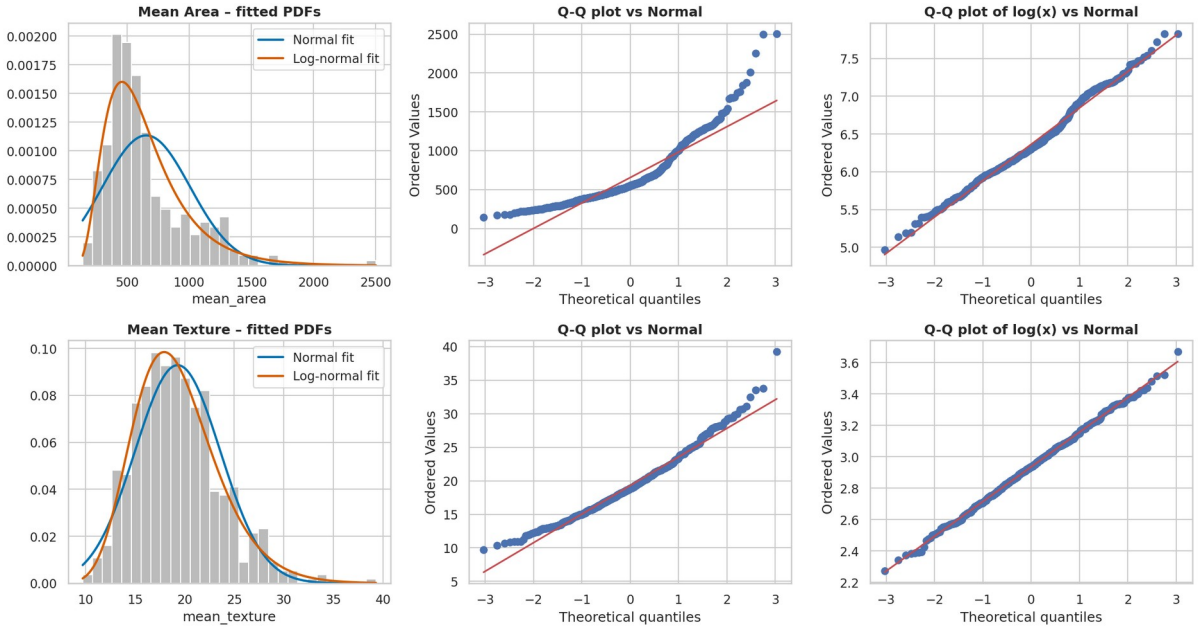


Figure 8: *Histogram with fitted Normal and Log-normal densities (left) and Q–Q plots of the raw and log-transformed values (centre, right).*

What it shows: a histogram of mean area and mean texture with fitted Normal and Log-normal probability density functions, plus quantile–quantile plots of the raw and log-transformed data.

Why it was created: to identify the theoretical continuous distribution that best describes each feature and to justify the transformation.

Insights obtained: Neither feature is normal: the raw-data Q–Q plot bends away from the line in the upper tail (right skew). The log-normal density follows the histogram much more closely, and the Q–Q plot of log(x) is close to a straight line. Table 7 confirms this for six features: Shapiro–Wilk rejects normality for all of them ($p < 0.001$), and the log-normal model has the lower AIC and the lower Kolmogorov–Smirnov statistic every time (for example mean area: AIC 8290 → 8031, KS 0.156 → 0.063). Part of the non-normality comes from mixing two populations (benign and malignant), so the log-normal fit is an approximation.

Table 7: *Normal versus Log-normal fit for six features (AIC and KS: lower is better)*

Feature	Skew	Shapiro p	AIC Normal	AIC Log-n.	KS Normal	KS Log-n.	Best fit
mean radius	0.94	< 0.001	3051	2966	0.113	0.066	Log-normal
mean texture	0.65	< 0.001	3278	3239	0.049	0.017	Log-normal
mean area	1.64	< 0.001	8290	8031	0.156	0.063	Log-normal
mean smoothness	0.46	< 0.001	-3235	-3250	0.036	0.026	Log-normal
mean compactness	1.19	< 0.001	-1729	-1893	0.101	0.031	Log-normal
area error	5.43	< 0.001	5962	5103	0.233	0.108	Log-normal

Visual 9 – Binomial distribution of malignant cases in a random sample (discrete)

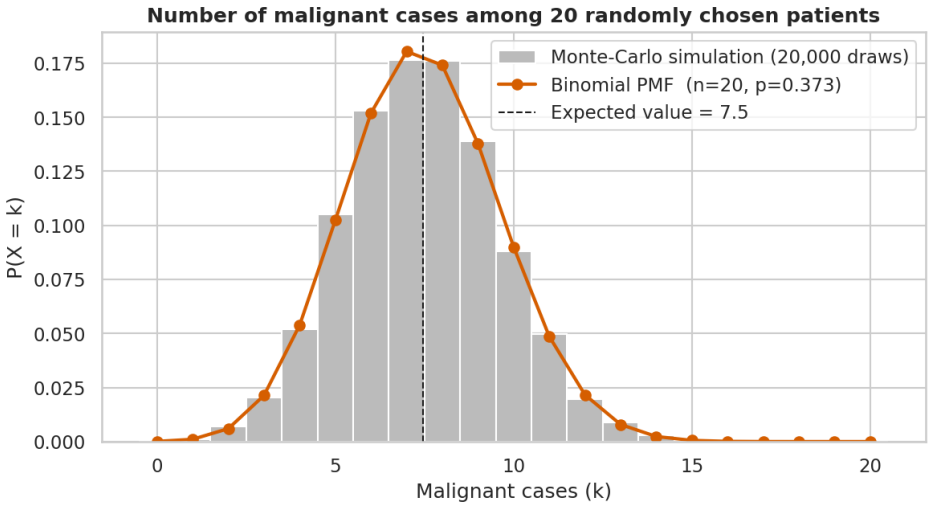


Figure 9: *Binomial probability mass function versus a 20,000-draw Monte-Carlo simulation.*

What it shows: the probability of seeing k malignant cases among n = 20 randomly chosen patients, modelled as Binomial(n = 20, p = 0.3726), overlaid with a Monte-Carlo simulation.

Why it was created: the diagnosis is a discrete (Bernoulli) variable, so the number of malignant cases in a sample is a discrete distribution whose shape can be predicted and verified.

Insights obtained: The expected count is 7.45 with a standard deviation of 2.16. The chance of at least 10 malignant cases is 17.1% and of at most 3 only 2.9%. The simulated histogram matches the theoretical PMF closely, validating the model. The assumptions – independent patients and a fixed probability – are an approximation, and the 37% prevalence reflects a diagnostic sample, not the general population.

4.4 Multivariate structure and refinement

Visual 10 – Principal component analysis (scree plot and 2-D projection)

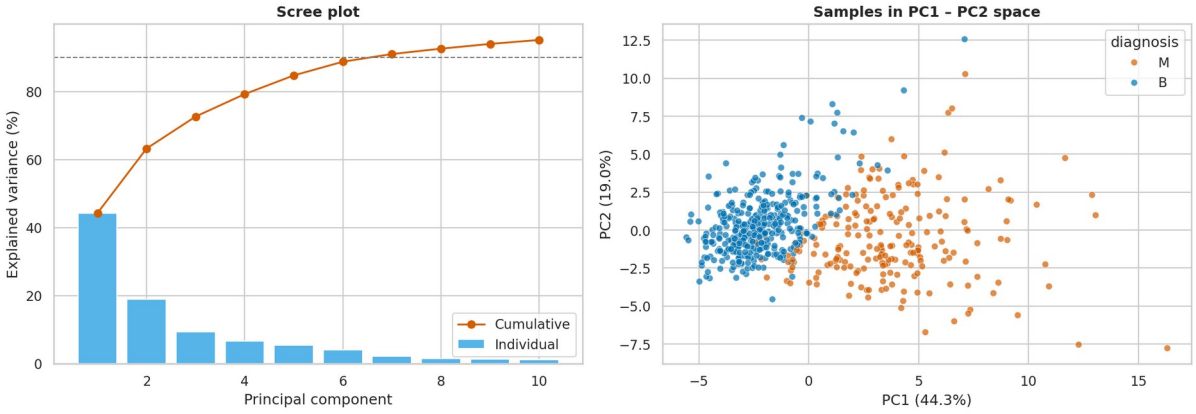


Figure 10: *PCA of the standardised 30 features: scree plot (left) and samples in PC1–PC2 space (right).*

What it shows: the variance explained by each principal component and the data projected onto the first two components.

Why it was created: to measure how many independent dimensions the 30 correlated features really contain and to see class structure in two dimensions.

Insights obtained: PC1 explains 44.3% and PC2 19.0% of the variance (63.3% together); 7 components retain 90% and 10 retain 95%, so the data are effectively about 7–10-dimensional. In the PC1–PC2 plane the benign and malignant clouds are clearly separated along PC1 with only a modest overlap.

Visual 11 – Skewness before and after the power transform

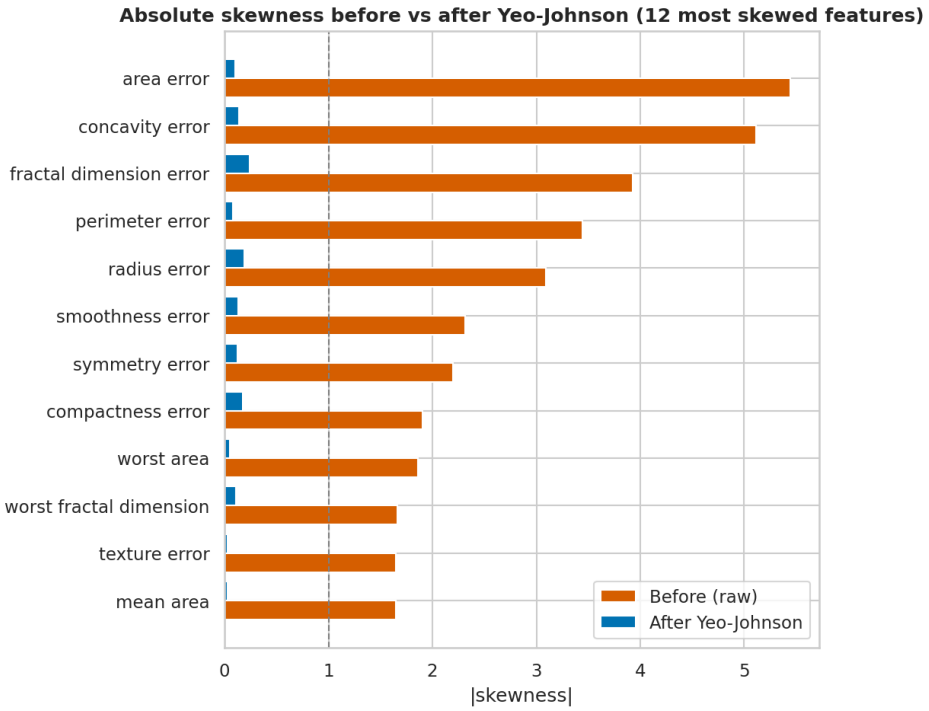


Figure 11: *Absolute skewness of the 12 most skewed features before and after the Yeo-Johnson transform.*

What it shows: the absolute skewness of the twelve most skewed features, raw versus transformed.

Why it was created: this is the 'Refine' stage – it checks that the preprocessing really fixed the problem identified during exploration.

Insights obtained: Mean absolute skewness falls from 1.74 to 0.25; for instance `area_error` drops from 5.45 to 0.1 and `perimeter_error` from 3.44 to 0.08. No feature is left above the |skew| = 1 threshold. Symmetric inputs make gradient-based training better conditioned and reduce the leverage of extreme values.

4.5 Interaction: interactive feature explorer (stage 7)

To complete the seven stages, the project includes a self-contained interactive visualization (`interactive/scatter_explorer.html`, written in plain HTML/JavaScript with no external libraries, so it works offline and on GitHub Pages). The user can choose any two of the 30 features for the axes, show or hide benign and malignant cases, and hover over a point to read the patient's values; the Pearson correlation of the visible points is recomputed live (Figure 12).

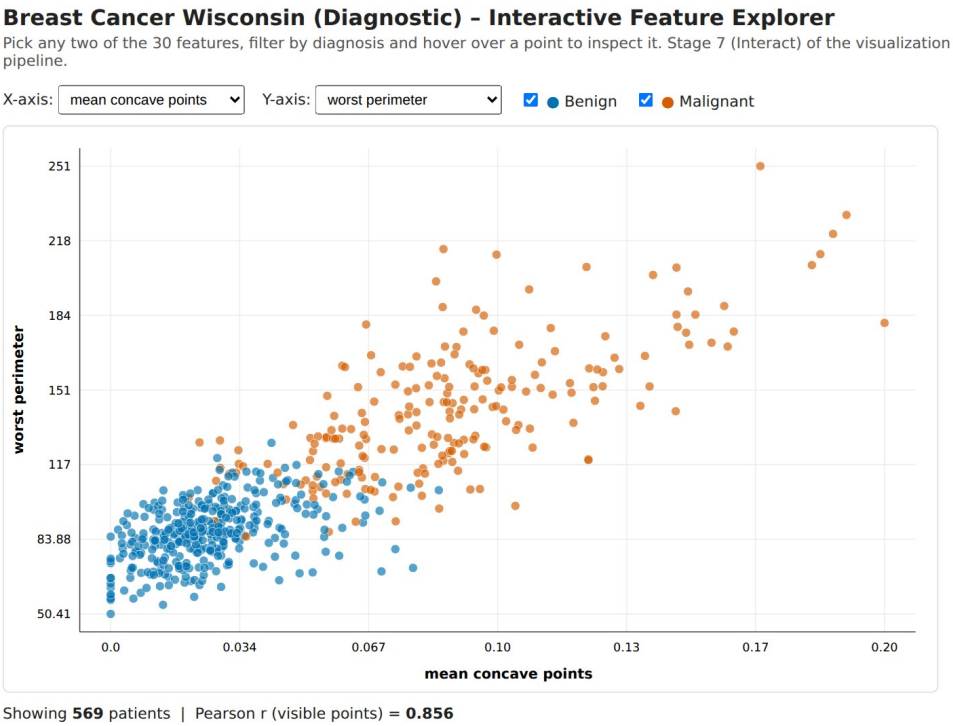


Figure 12: *Screenshot of the interactive explorer (mean concave points versus worst perimeter, r = 0.856).*

Insight: interaction lets the reader test the hypotheses of this report on their own – for example, switching the axes to mean texture and mean smoothness immediately reveals the heavy class overlap seen in Visual 6, while worst perimeter against worst concave points reproduces the near-separable clusters.

5. Deep Learning Model

5.1 Choice of model

The features are tabular and numeric, so a **multi-layer perceptron (MLP)** – a fully connected deep feed-forward neural network – is the appropriate deep-learning architecture (convolutional and recurrent networks are intended for images and sequences). The MLP learns non-linear combinations of the 30 features through several hidden layers. It was implemented with scikit-learn's `MLPClassifier`, trained epoch by epoch with `partial_fit` so that training and validation loss and accuracy could be recorded after every epoch, and compared with two baselines: logistic regression and a 300-tree random forest.

5.2 Architecture

Table 8: *Network architecture (30 → 64 → 32 → 16 → 1)*

Layer	Units	Activation	Trainable parameters
Input	30	–	0
Hidden layer 1 (dense)	64	ReLU	30 × 64 + 64 = 1,984
Hidden layer 2 (dense)	32	ReLU	64 × 32 + 32 = 2,080
Hidden layer 3 (dense)	16	ReLU	32 × 16 + 16 = 528
Output (dense)	1	Sigmoid → P(malignant)	16 × 1 + 1 = 17
Total			**4,609**

5.3 Training parameters

Table 9: *Training configuration of the final model*

Parameter	Value
Loss function	Binary cross-entropy (log-loss)
Optimiser	Adam (β_1 = 0.9, β_2 = 0.999)
Learning rate	0.001
Mini-batch size	32
L2 weight penalty (alpha)	0.01
Maximum epochs / early stopping	200 / stop after 20 epochs without improvement of the validation loss; best weights restored
Epochs actually run / best epoch	32 / 12
Inputs	30 Yeo-Johnson-transformed, standardised features (fitted on train only)
Data used	train 398 (fit) · validation 85 (tuning, early stopping) · test 86 (final evaluation, used once)
Random seed	42 (data split, weights initialisation, shuffling)

5.4 Hyper-parameter tuning

A grid search of 24 configurations was run on the validation set: four architectures – (32, 16), (64, 32), (64, 32, 16) and (128, 64, 32) – × two L2 penalties (0.0001, 0.01) × three learning rates

(0.00025, 0.0005, 0.001). Every architecture has at least two hidden layers. Configurations were ranked by validation accuracy, ties broken by validation loss. A learning rate of 0.005 was tried first, but the validation loss then reached its minimum after only about two epochs, which is unstable and gives uninformative learning curves, so the grid was centred on gentler rates (this choice was made on training behaviour, not on test results).

Table 10: *Top-5 configurations by validation performance*

Rank	Hidden layers	alpha	Learning rate	Best epoch	Val. loss	Val. accuracy	Val. AUC
1	(64, 32, 16)	0.01	0.001	12	0.0595	98.82%	0.9988
2	(64, 32, 16)	0.0001	0.001	12	0.0595	98.82%	0.9988
3	(64, 32)	0.01	0.001	17	0.0642	98.82%	0.9959
4	(64, 32)	0.0001	0.001	17	0.0645	98.82%	0.9959
5	(64, 32)	0.01	0.0005	34	0.0652	98.82%	0.9959

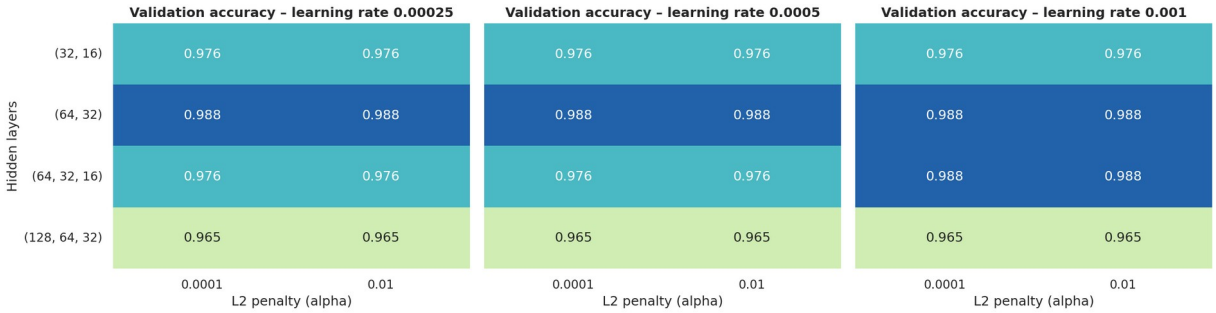


Figure 13: *Validation accuracy for all 24 configurations (one panel per learning rate).*

What it shows: the validation accuracy of every architecture × alpha × learning-rate combination as a heat-map.

Why it was created: to choose hyper-parameters on data the network is not trained on, and to see how sensitive the result is.

Insights obtained: Validation accuracy ranges only from 96.5% to 98.8%, i.e. one or two patients out of 85, so the landscape is flat and differences must not be over-interpreted. Even so, the two medium-sized networks, (64, 32) and (64, 32, 16), reach the highest validation accuracy (98.8%), while the smallest (32, 16) tops out at 97.6% and the widest (128, 64, 32) at 96.5%, probably because it over-fits the 398 training patients. The L2 penalty has no visible effect at this scale (mean validation accuracy 97.7% for both values). The best configuration is (64, 32, 16) with alpha = 0.01 and learning rate 0.001.

5.5 Feature analysis and feature importance

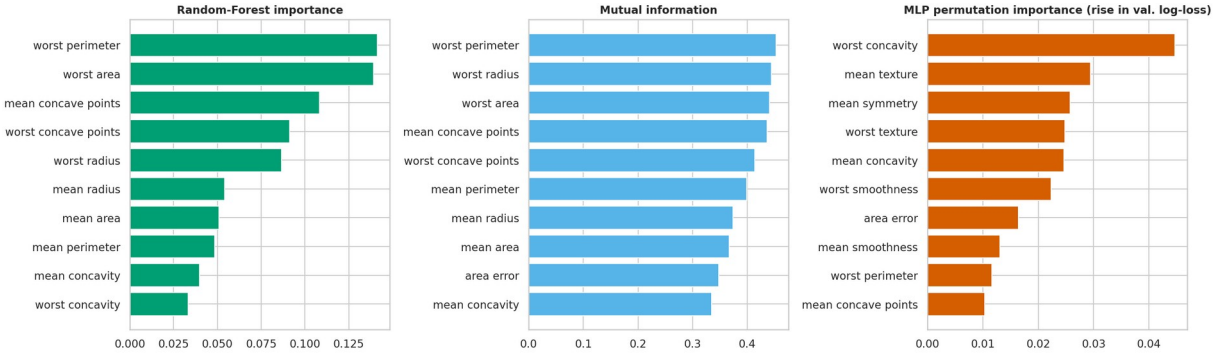


Figure 14: *Top-10 features by Random-Forest importance, mutual information and permutation importance of the trained MLP.*

What it shows: three independent rankings of feature usefulness: impurity-based importance of a random forest, mutual information with the target, and the rise in validation log-loss of the MLP when one feature is randomly shuffled (permutation importance).

Why it was created: to understand which measurements drive the diagnosis and to check that the neural network relies on sensible features.

Insights obtained: The random forest and mutual information agree: worst perimeter, worst area, worst radius, mean concave points and worst concave points are the most informative (RF importance 0.141 for worst perimeter; mutual information 0.453). These are exactly the large-effect features of Table 6. Permutation importance of the MLP ranks different features first (worst concavity, mean texture, mean symmetry) because the size features are so redundant (correlations above 0.95): shuffling one of them hardly hurts the network, as its correlated twins compensate, whereas texture and symmetry carry information that nothing else duplicates. The two views are therefore complementary – ‘strongest signal’ versus ‘unique signal’.

6. Result Visualization and Interpretation

6.1 Learning curves: loss and accuracy versus epoch

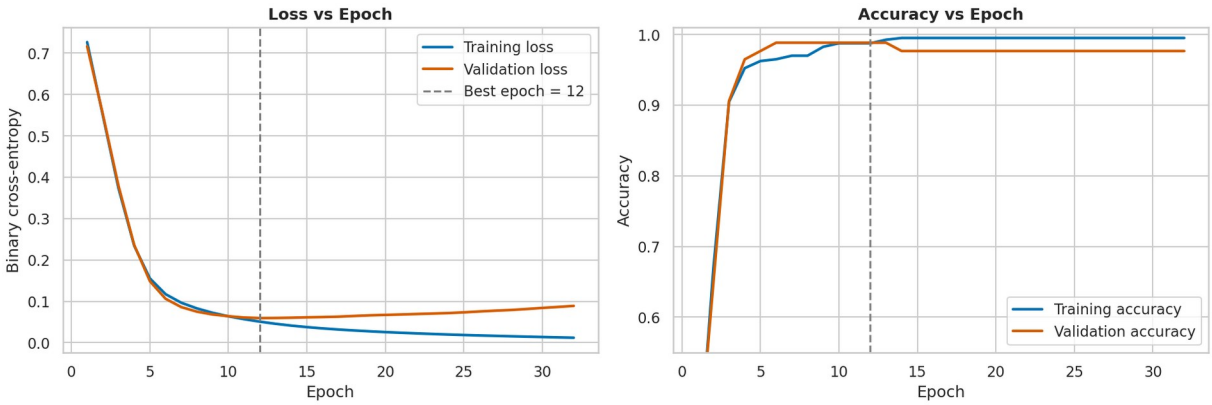


Figure 15: *Training and validation loss (left) and accuracy (right) per epoch; dashed line = epoch of best validation loss.*

What it shows: the binary cross-entropy loss and the accuracy of the network on the training and validation sets after each epoch.

Why it was created: learning curves reveal convergence speed, over-fitting and the right moment to stop training.

Insights obtained: Both losses fall together during the first ~10 epochs, from about 0.72 to 0.06. The validation loss reaches its minimum (0.059) at epoch 12; after that the training loss keeps decreasing towards 0.01 while the validation loss slowly rises to about 0.09 – the classical signature of over-fitting. Validation accuracy peaks at 98.8% (84 of 85 patients) and later slips to 97.6%, while training accuracy reaches 99.5%. Early stopping (patience 20) halted training at epoch 32 and restored the weights of epoch 12, preventing this degradation.

6.2 Test-set performance and baseline comparison

The test set (86 patients, 32 malignant) was used once, after all choices had been made on the validation set. Positive class = malignant; the decision threshold is 0.5.

Table 11: *Test-set results (FP = false positives, FN = false negatives)*

Model	Accuracy	Precision	Recall	Specificity	F1	ROC-AUC	Log-loss	FP / FN
MLP (deep NN)	98.84%	100.0%	96.9%	100.0%	98.4%	1.0000	0.039	0 / 1
Logistic Regression	97.67%	96.9%	96.9%	98.2%	96.9%	0.9988	0.046	1 / 1
Random Forest	98.84%	100.0%	96.9%	100.0%	98.4%	0.9986	0.082	0 / 1

The MLP classified 85 of 86 test patients correctly (98.84%), with no false alarms (precision 100%) and one missed malignant tumour (recall 96.9%). It matches the random forest in accuracy, beats logistic regression by one patient, and has the best ROC-AUC and the lowest log-loss. Because 0 or 1 errors in 86 cases is a noisy estimate (even a perfect score would only bound the true error rate below about 3.5% with 95% confidence), the more reliable figure is the **5-fold stratified cross-validation** on train + validation (preprocessing refitted inside each

fold): accuracy **96.5% ± 3.0%**, AUC **0.992 ± 0.006**. The honest summary is therefore ‘about 96–99% accuracy’, with all three models statistically indistinguishable on a dataset of this size.

6.3 Confusion matrix

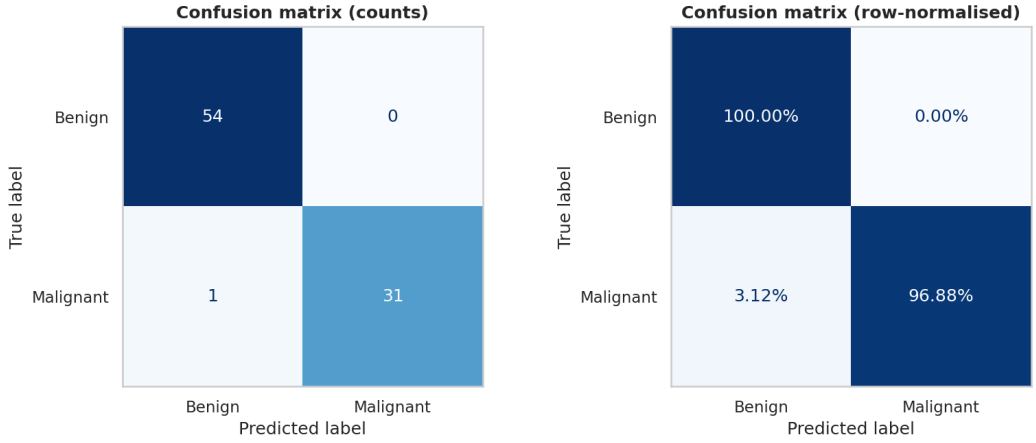


Figure 16: *Confusion matrix of the MLP on the test set (counts and row-normalised).*

What it shows: how the 86 test patients were distributed over true and predicted classes.

Why it was created: the confusion matrix shows which type of error the model makes, which accuracy hides. In cancer screening a false negative (missed tumour) is far more costly than a false positive.

Insights obtained: All 54 benign tumours were recognised (specificity 100%) and 31 of 32 malignant tumours were detected (recall 96.9%). The single error is a malignant tumour predicted as benign, i.e. the costly kind. A clinical deployment would lower the decision threshold – chosen on validation data, not on the test set – to trade a few extra false alarms for higher recall.

6.4 ROC curve, AUC and precision–recall curve

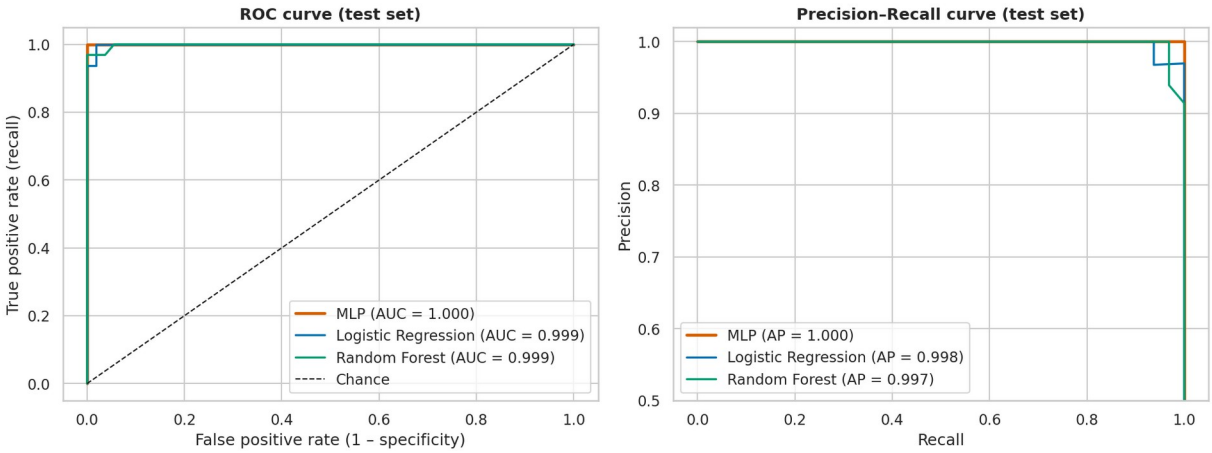


Figure 17: *ROC curves with AUC (left) and precision–recall curves with average precision (right) for the MLP, logistic regression and random forest.*

What it shows: the trade-off between true-positive rate and false-positive rate over all possible thresholds (ROC), and between precision and recall (PR), for the three models.

Why it was created: ROC–AUC and PR curves evaluate the ranking quality of the predicted probabilities independently of the 0.5 threshold, and PR curves are more informative under class imbalance.

Insights obtained: The MLP's curve hugs the top-left corner (AUC = 1.000, average precision 1.000): on the test set every malignant patient received a higher probability than every benign one (the AUC is exactly 1.0 on these 86 patients). Logistic regression (AUC 0.9988) and the random forest (0.9986) are almost as good. Perfect ranking but one error at threshold 0.5 means that the model discriminates perfectly here and the single miss is a threshold-calibration matter rather than a lack of discriminative power. The cross-validated AUC of 0.992 is the more cautious estimate.

6.5 Error analysis



Figure 18: *Distribution of predicted malignancy probability by true class (left) and of the prediction error (right).*

What it shows: the histogram of predicted probabilities for each true class (left) and the distribution of the residual, true label minus predicted probability (right).

Why it was created: for a classifier the error distribution shows how confident and how well-calibrated the predictions are and where the errors sit.

Insights obtained: The predictions are strongly bimodal: 86% of the test patients receive a probability below 0.05 or above 0.95. The residuals are centred at 0.008 with a standard deviation of 0.0931, i.e. no systematic bias. The only misclassified patient is a malignant tumour given P(malignant) = 0.37 – an uncertain, not a confidently wrong, prediction – and a few others lie between about 0.15 and 0.8, which would be natural candidates for human review. There are no confident errors.

7. Conclusion and Future Scope

7.1 What was learned from the dataset

The WDBC data demonstrate how much can be learned from careful exploration before modelling: the variables are clean but skewed, heavily redundant and unevenly informative. A handful of size and concavity measurements carry most of the diagnostic signal, while texture, smoothness, symmetry and fractal dimension contribute little on their own.

7.2 Key patterns and findings

- Malignant tumours have larger nuclei with more concave contours: the eight strongest features have Cohen's d between 2.1 and 2.7, and 26 of 30 features differ significantly after Bonferroni correction.
- The classes are almost linearly separable in a combination of size and concavity features (pair plot, PCA), which explains the high performance of every model.
- The features are highly collinear: 21 of 435 pairs have $|r| > 0.9$, and seven principal components capture 90% of the variance.
- Most features are right-skewed and better described by a log-normal than by a normal distribution; a Yeo-Johnson transform reduces mean absolute skewness from 1.74 to 0.25. The number of malignant cases in a random sample follows a binomial distribution.
- Outliers are predominantly malignant and physiologically meaningful, so they should be kept rather than removed.

7.3 How the model performed

The 30–64–32–16–1 MLP reached 98.8% accuracy, 96.9% recall, 100% precision and ROC-AUC ≈ 1.00 on the held-out test set, and 96.5% ± 3.0% accuracy (AUC 0.992) in five-fold cross-validation. Its performance is on par with random forest and slightly better than logistic regression; with only 569 records the differences are within sampling noise, which is itself an important finding – a deeper network does not automatically beat a simple one on small tabular data.

7.4 Challenges faced

- **Small sample:** a test set of 86 patients can change accuracy by 1.2 points with a single case, so cross-validation was added.
- **Skewness:** the first transformation (`log1p`) was ineffective for small-valued features; replacing it with Yeo-Johnson solved the problem.
- **Multicollinearity:** correlated features make permutation importance and linear coefficients hard to interpret.
- **Training stability:** a high learning rate (0.005) made the network peak after two epochs; the grid was re-centred on gentler rates, and early stopping was necessary to avoid over-fitting.
- **Data leakage:** scaling and transformation must be fitted on the training set only and re-fitted inside every cross-validation fold.

7.5 Limitations

The data are pre-extracted nuclear measurements from a single source and period, and may not generalise to other laboratories or imaging protocols. The binomial model assumes independent patients and a fixed prevalence. The network was built with scikit-learn on a CPU; no external validation set was available. The model is an educational exercise and **not a clinical diagnostic tool**.

7.6 Future scope and suggested improvements

- **Threshold optimisation and calibration:** choose the decision threshold on validation data to guarantee a minimum recall, and apply Platt or isotonic calibration so that probabilities can be trusted.
- **Better validation:** repeated or nested cross-validation and bootstrap confidence intervals for every metric; external validation on an independent breast-cancer dataset.
- **Model improvements:** dropout and batch normalisation in a TensorFlow/Keras or PyTorch implementation, ensembles of several seeds, gradient-boosted trees as an additional baseline, and feature selection or PCA to remove redundancy.
- **Explainability:** SHAP or LIME explanations for individual predictions, so that a pathologist can see which measurements drove the result.
- **Additional features and data:** patient age, family history, imaging findings and larger multi-centre samples; or learning directly from the raw FNA images with a convolutional neural network.
- **Deployment:** publish the interactive explorer on GitHub Pages and wrap the model in a small web application (Streamlit) for demonstration.

8. References

(APA 7th edition style, in alphabetical order)

Dua, D., & Graff, C. (2019). *UCI Machine Learning Repository*. University of California, Irvine, School of Information and Computer Sciences. https://archive.ics.uci.edu

Fry, B. (2007). *Visualizing data: Exploring and explaining data with the Processing environment*. O'Reilly Media.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95.

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., … Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Street, W. N., Wolberg, W. H., & Mangasarian, O. L. (1993). Nuclear feature extraction for breast tumor diagnosis. *Proceedings of IS&T/SPIE International Symposium on Electronic Imaging: Science and Technology, 1905*, 861–870.

Tukey, J. W. (1977). *Exploratory data analysis*. Addison-Wesley.

Waskom, M. L. (2021). seaborn: Statistical data visualization. *Journal of Open Source Software, 6*(60), 3021.

Appendix A – Code Section

The complete, commented Python source (`src/eda_mlp_pipeline.py`) is reproduced below. It covers data loading, EDA, preprocessing, visualization, model creation and training, hyper-parameter tuning and evaluation. The same code, with executed outputs, is in `notebooks/EDA_Visualization_MLP.ipynb`. The long HTML/JavaScript template of the interactive explorer is abbreviated here; it is complete in the repository.

```
# %% [markdown]
# # Exploratory Data Analysis & Visualization of the Breast Cancer Wisconsin (Diagnostic) Dataset
# **Course:** U21ADP05 – Exploratory Data Analysis and Visualization | **Assignment II – Case Study**
#
# This notebook covers the full project flow:
# 1. Acquire → 2. Parse → 3. Filter (clean) → 4. Mine (statistics) → 5. Represent (visuals) → 6. Refine →
7. Interact
# followed by deep-learning model building (MLP), hyper-parameter tuning, and evaluation visualisations.
#
# *Run order: top to bottom. All figures are saved into `figures/`, tables into `outputs/`.*

# %%
# ---------------------------------------------------------------- 0. SETUP
import os, json, copy, warnings
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib
IN_NB = "get_ipython" in globals()          # True when running inside Jupyter
if not IN_NB:
    matplotlib.use("Agg")                    # headless backend when run as a plain .py script
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.preprocessing import StandardScaler, PowerTransformer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import make_pipeline
from sklearn.decomposition import PCA
from sklearn.neural_network import MLPClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import mutual_info_classif
from sklearn.inspection import permutation_importance
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_auc_score, roc_curve, precision_recall_curve,
                             average_precision_score, confusion_matrix, log_loss,
                             ConfusionMatrixDisplay)

warnings.filterwarnings("ignore")
try:
    from IPython.display import display
except ImportError:                          # plain-python fallback
    display = print

SEED = 42
np.random.seed(SEED)

# ---- project paths (works from the repo root, notebooks/ or src/) ----
ROOT = Path.cwd()
if ROOT.name in ("notebooks", "src"):
    ROOT = ROOT.parent
DATA, PROC, FIG, OUT, INTER = (ROOT / "data", ROOT / "data" / "processed",
                               ROOT / "figures", ROOT / "outputs", ROOT / "interactive")
for p in (DATA, PROC, FIG, OUT, INTER):
```

```
    p.mkdir(parents=True, exist_ok=True)

# ---- consistent, colour-blind-safe visual style (Okabe-Ito palette) ----
PALETTE = {"B": "#0072B2", "M": "#D55E00"}   # Benign = blue, Malignant = vermillion
sns.set_theme(style="whitegrid", context="notebook", font_scale=1.0)
plt.rcParams.update({"figure.dpi": 100, "axes.titleweight": "bold", "axes.titlesize": 12})
pd.set_option("display.width", 200); pd.set_option("display.max_columns", 14)
metrics = {}                                 # collects every number reported in the project report


def finish(name):
    """Save the current figure into figures/ and display it."""
    plt.tight_layout()
    plt.savefig(FIG / name, dpi=150, bbox_inches="tight")
    plt.show() if IN_NB else plt.close()


# %% [markdown]
# ## STAGE 1 – ACQUIRE
# **Dataset:** Breast Cancer Wisconsin (Diagnostic) – UCI Machine Learning Repository (Wolberg, Street &
Mangasarian).
# Features are computed from digitised images of fine-needle-aspirate (FNA) of breast masses and describe
the cell nuclei.
# The CSV is stored in `data/`; if it is missing it is regenerated from scikit-learn's bundled copy of the
same UCI data.

# %%
CSV = DATA / "breast_cancer_wisconsin.csv"
if not CSV.exists():
    raw = load_breast_cancer(as_frame=True).frame
    raw.columns = [c.replace(" ", "_") for c in raw.columns]
    # scikit-learn encodes target 0 = malignant, 1 = benign -> convert to the original M / B labels
    raw["diagnosis"] = np.where(raw["target"] == 0, "M", "B")
    raw = raw.drop(columns="target")
    raw.to_csv(CSV, index=False)
df = pd.read_csv(CSV)
print("Loaded:", CSV.name, "| shape =", df.shape)
display(df.head())

# %% [markdown]
# ## STAGE 2 – PARSE (understand structure & data types)

# %%
FEATURES = [c for c in df.columns if c != "diagnosis"]
print(f"Rows: {df.shape[0]} | Features: {len(FEATURES)} | Target: 'diagnosis'")
print("\nData types:\n", df.dtypes.value_counts())
base = ["radius", "texture", "perimeter", "area", "smoothness",
        "compactness", "concavity", "concave_points", "symmetry", "fractal_dimension"]
groups = {"mean": [f"mean_{b}" for b in base],
          "error": [f"{b}_error" for b in base],
          "worst": [f"worst_{b}" for b in base]}
print("\nFeature groups: 10 'mean' + 10 'error' (standard error) + 10 'worst' (largest value)
measurements")
desc = df[FEATURES].describe().T
display(desc[["mean", "std", "min", "50%", "max"]].round(3).head(12))
metrics["n_rows"], metrics["n_features"] = int(df.shape[0]), len(FEATURES)

# %% [markdown]
# ## STAGE 3 – FILTER (missing values, duplicates, outliers, cleaning)

# %%
# ---- 3a. Missing values & duplicates ----
n_missing = int(df.isna().sum().sum())
n_dupes = int(df.duplicated().sum())
print("Total missing values :", n_missing)
print("Duplicate rows       :", n_dupes)
metrics["missing_values"], metrics["duplicate_rows"] = n_missing, n_dupes
# Cleaning pipeline (kept generic so it also works on dirtier data): drop duplicates, median-impute
numerics
df = df.drop_duplicates().reset_index(drop=True)
```

```
df[FEATURES] = df[FEATURES].fillna(df[FEATURES].median())
print("Constant (zero-variance) features:", [c for c in FEATURES if df[c].nunique() == 1])

# ---- 3b. Outlier detection (Tukey IQR rule: outside Q1-1.5*IQR .. Q3+1.5*IQR) ----
Q1, Q3 = df[FEATURES].quantile(0.25), df[FEATURES].quantile(0.75)
IQR = Q3 - Q1
is_out = (df[FEATURES] < Q1 - 1.5 * IQR) | (df[FEATURES] > Q3 + 1.5 * IQR)
out_pct = (is_out.mean() * 100).sort_values(ascending=False)
rows_with_out = int(is_out.any(axis=1).sum())
print(f"Rows with at least one IQR outlier: {rows_with_out} ({rows_with_out/len(df):.1%})")
display(out_pct.head(8).round(2).to_frame("outlier %"))
metrics["rows_with_outliers"] = rows_with_out
metrics["top_outlier_features"] = {k: round(float(v), 2) for k, v in out_pct.head(5).items()}

# Do outliers concentrate in one class? (decides whether they are errors or genuine biology)
share_M = df.loc[is_out.any(axis=1), "diagnosis"].value_counts(normalize=True).get("M", 0)
print(f"Share of outlier rows that are malignant: {share_M:.1%}")
metrics["outlier_rows_malignant_share"] = round(float(share_M), 3)
# DECISION: outliers are KEPT. They are physiologically plausible (very large malignant nuclei), not entry
errors;
# they are tamed instead by a power transform + standardisation (Stage 6 / preprocessing below).

# %% [markdown]
# ## STAGE 4 – MINE (statistics, distributions, relationships)
# ### 4.1 Univariate summary, skewness and class balance

# %%
cls_counts = df["diagnosis"].value_counts()
p_mal = cls_counts["M"] / len(df)
print(cls_counts.to_dict(), f"-> malignant share = {p_mal:.3f}")
metrics.update(n_benign=int(cls_counts["B"]), n_malignant=int(cls_counts["M"]),
p_malignant=round(float(p_mal), 4))

skew = df[FEATURES].skew().sort_values(ascending=False)
metrics["n_skewed_gt1"] = int((skew.abs() > 1).sum())
metrics["most_skewed"] = {k: round(float(v), 2) for k, v in skew.head(5).items()}
print("Features with |skewness| > 1 :", metrics["n_skewed_gt1"], "of", len(FEATURES))
display(skew.head(6).round(2).to_frame("skewness"))

# %% [markdown]
# ### 4.2 Group comparison: do benign and malignant tumours differ? (Mann-Whitney U + Cohen's d)

# %%
B, M = df[df.diagnosis == "B"], df[df.diagnosis == "M"]
rows = []
for f in FEATURES:
    u, pval = stats.mannwhitneyu(M[f], B[f], alternative="two-sided")
    pooled = np.sqrt(((len(M) - 1) * M[f].var() + (len(B) - 1) * B[f].var()) / (len(M) + len(B) - 2))
    rows.append((f, M[f].mean(), B[f].mean(), (M[f].mean() - B[f].mean()) / pooled, pval))
effect = pd.DataFrame(rows, columns=["feature", "mean_malignant", "mean_benign", "cohens_d", "p_value"])
effect = effect.reindex(effect.cohens_d.abs().sort_values(ascending=False).index).reset_index(drop=True)
effect.to_csv(OUT / "feature_effect_sizes.csv", index=False)
display(effect.head(8).round(4))
metrics["n_sig_features_p05"] = int((effect.p_value < 0.05).sum())
metrics["top_effect"] = effect.head(8).round(3).to_dict("records")
metrics["least_effect"] = effect.tail(3).round(3).to_dict("records")

# %% [markdown]
# ### 4.3 Probability distributions – continuous (Normal vs Log-normal fit) and discrete (Binomial)

# %%
fit_rows = []
for f in ["mean_radius", "mean_texture", "mean_area", "mean_smoothness", "mean_compactness", "area_error"]:
    x = df[f].values
    mu, sd = stats.norm.fit(x)
    shape, loc, scale = stats.lognorm.fit(x, floc=0)
    ll_n = stats.norm.logpdf(x, mu, sd).sum()
    ll_l = stats.lognorm.logpdf(x, shape, loc, scale).sum()
    ks_n = stats.kstest(x, "norm", args=(mu, sd)).statistic
    ks_l = stats.kstest(x, "lognorm", args=(shape, loc, scale)).statistic
```

```
    aic_n, aic_l = 2 * 2 - 2 * ll_n, 2 * 2 - 2 * ll_l          # AIC = 2k - 2 ln(L), k = 2 parameters
    fit_rows.append(dict(feature=f, skew=round(float(stats.skew(x)), 2),
shapiro_p=float(stats.shapiro(x).pvalue),
                         aic_normal=round(float(aic_n), 1), aic_lognormal=round(float(aic_l), 1),
                         ks_normal=round(float(ks_n), 3), ks_lognormal=round(float(ks_l), 3),
                         best_fit="Log-normal" if aic_l < aic_n else "Normal"))
fit_df = pd.DataFrame(fit_rows)
fit_df.to_csv(OUT / "distribution_fit_summary.csv", index=False)
display(fit_df)
metrics["dist_fit"] = fit_df.to_dict("records")

# Discrete distribution: number of malignant cases among n = 20 randomly chosen patients ~ Binomial(n, p)
n_trial = 20
k = np.arange(0, n_trial + 1)
pmf = stats.binom.pmf(k, n_trial, p_mal)
sim = np.random.default_rng(SEED).binomial(n_trial, p_mal, 20000)   # Monte-Carlo check
metrics["binom"] = dict(n=n_trial, p=round(float(p_mal), 4), mean=round(n_trial * p_mal, 2),
                        sd=round(float(np.sqrt(n_trial * p_mal * (1 - p_mal))), 2),
                        p_ge_10=round(float(stats.binom.sf(9, n_trial, p_mal)), 4),
                        p_le_3=round(float(stats.binom.cdf(3, n_trial, p_mal)), 4))
print(metrics["binom"])

# %% [markdown]
# ### 4.4 Multivariate structure: correlation & PCA

# %%
corr = df[FEATURES].corr()
pairs = corr.where(np.triu(np.ones(corr.shape, dtype=bool), 1)).stack().dropna()
metrics["n_pairs_abs_r_gt_90"] = int((pairs.abs() > 0.9).sum())
metrics["n_pairs_total"] = int(len(pairs))
metrics["strongest_pairs"] = [(a, b, round(float(corr.loc[a, b]), 3))
                              for (a, b) in pairs.abs().sort_values(ascending=False).head(4).index]
y_bin = (df.diagnosis == "M").astype(int)
target_corr = df[FEATURES].corrwith(y_bin).sort_values(key=abs, ascending=False)
metrics["top_target_corr"] = {k: round(float(v), 3) for k, v in target_corr.head(5).items()}
print("Pairs with |r| > 0.9:", metrics["n_pairs_abs_r_gt_90"], "of", metrics["n_pairs_total"])
print(metrics["strongest_pairs"])

pca_full = PCA().fit(StandardScaler().fit_transform(df[FEATURES]))
cum = np.cumsum(pca_full.explained_variance_ratio_)
metrics["pca"] = dict(pc1=round(float(pca_full.explained_variance_ratio_[0]), 3),
                      pc2=round(float(pca_full.explained_variance_ratio_[1]), 3),
                      n_for_90=int(np.argmax(cum >= 0.90) + 1), n_for_95=int(np.argmax(cum >= 0.95) + 1))
print(metrics["pca"])

# %% [markdown]
# ## STAGE 5 – REPRESENT (visualisations)
# Each figure below is explained (*What it shows / Why created / Insight*) in the project report.
# ### Visual 1 – Class distribution

# %%
fig, ax = plt.subplots(1, 2, figsize=(10, 4))
order = ["B", "M"]
bars = ax[0].bar(["Benign (B)", "Malignant (M)"], [cls_counts[o] for o in order], color=[PALETTE[o] for o
in order])
for b_, o in zip(bars, order):
    ax[0].text(b_.get_x() + b_.get_width() / 2, b_.get_height() + 4,
f"{cls_counts[o]}\n({cls_counts[o]/len(df):.1%})", ha="center")
ax[0].set(title="Class distribution of the target", ylabel="Number of patients", ylim=(0, 430))
ax[1].pie([cls_counts[o] for o in order], labels=["Benign", "Malignant"], colors=[PALETTE[o] for o in
order],
          autopct="%1.1f%%", startangle=90, wedgeprops=dict(edgecolor="white"))
ax[1].set_title("Class proportion")
finish("fig01_class_distribution.png")

# %% [markdown]
# ### Visual 2 – Histograms with KDE (univariate distributions, split by diagnosis)

# %%
```

```
key6 = ["mean_radius", "mean_texture", "mean_area", "mean_smoothness", "mean_concave_points",
"worst_perimeter"]
fig, axes = plt.subplots(2, 3, figsize=(14, 7))
for ax, f in zip(axes.ravel(), key6):
    sns.histplot(data=df, x=f, hue="diagnosis", palette=PALETTE, kde=True, stat="density",
common_norm=False,
                 alpha=0.45, bins=25, ax=ax)
    ax.set_title(f.replace("_", " ").title()); ax.set_xlabel("")
    ax.axvline(df[f].median(), color="grey", ls="--", lw=1)
fig.suptitle("Distribution of key features by diagnosis (dashed = overall median)", fontweight="bold",
y=1.02)
finish("fig02_histograms_kde.png")

# %% [markdown]
# ### Visual 3 – Box plots (spread and outliers)

# %%
fig, axes = plt.subplots(2, 3, figsize=(14, 7))
for ax, f in zip(axes.ravel(), key6):
    sns.boxplot(data=df, x="diagnosis", y=f, order=["B", "M"], palette=PALETTE, hue="diagnosis",
legend=False,
                width=0.55, fliersize=3, ax=ax)
    ax.set_title(f.replace("_", " ").title()); ax.set_xlabel(""); ax.set_ylabel("")
    ax.set_xticks([0, 1]); ax.set_xticklabels(["Benign", "Malignant"])
fig.suptitle("Box plots by diagnosis (points beyond whiskers = IQR outliers)", fontweight="bold", y=1.02)
finish("fig03_boxplots.png")

# %% [markdown]
# ### Visual 4 – Outlier prevalence (Tukey IQR) per feature

# %%
fig, ax = plt.subplots(figsize=(8, 6))
top_o = out_pct.head(15)[::-1]
ax.barh([i.replace("_", " ") for i in top_o.index], top_o.values, color="#56B4E9")
for i, v in enumerate(top_o.values):
    ax.text(v + 0.15, i, f"{v:.1f}%", va="center", fontsize=9)
ax.set(title="Top-15 features by share of IQR outliers", xlabel="% of observations flagged as outliers")
finish("fig04_outlier_prevalence.png")

# %% [markdown]
# ### Visual 5 – Correlation heat-map (multivariate relationships)

# %%
fig, axes = plt.subplots(1, 2, figsize=(17, 7), gridspec_kw={"width_ratios": [1.1, 1]})
mc = df[groups["mean"]].corr()
sns.heatmap(mc, annot=True, fmt=".2f", cmap="vlag", center=0, vmin=-1, vmax=1, square=True,
            cbar_kws={"shrink": .7}, annot_kws={"size": 7}, ax=axes[0])
axes[0].set_title("Correlation of the 10 'mean' features")
axes[0].set_xticklabels([t.get_text().replace("mean_", "") for t in axes[0].get_xticklabels()],
rotation=45, ha="right")
axes[0].set_yticklabels([t.get_text().replace("mean_", "") for t in axes[0].get_yticklabels()], rotation=0)
tc = target_corr.head(12)[::-1]
axes[1].barh([i.replace("_", " ") for i in tc.index], tc.values, color=["#D55E00" if v > 0 else "#0072B2"
for v in tc.values])
axes[1].set(title="Top-12 features by correlation with malignancy", xlabel="Pearson r with diagnosis (M =
1)")
finish("fig05_correlation_heatmap.png")

# %% [markdown]
# ### Visual 6 – Scatter plots (bivariate relationships)

# %%
fig, axes = plt.subplots(1, 3, figsize=(16, 4.8))
for ax, (xf, yf) in zip(axes, [("mean_radius", "mean_area"), ("mean_perimeter", "mean_concave_points"),
("mean_texture", "mean_smoothness")]):
    sns.scatterplot(data=df, x=xf, y=yf, hue="diagnosis", palette=PALETTE, alpha=0.65, s=30, ax=ax)
    r = df[xf].corr(df[yf])
    ax.set_title(f"{xf.replace('_',' ')} vs {yf.replace('_',' ')}  (r = {r:.2f})", fontsize=10.5)
metrics["scatter_r"] = {"radius_area": round(float(df.mean_radius.corr(df.mean_area)), 3),
```

```
                        "perimeter_concave": round(float(df.mean_perimeter.corr(df.mean_concave_points)),
3),
                        "texture_smoothness": round(float(df.mean_texture.corr(df.mean_smoothness)), 3)}
finish("fig06_scatter_relationships.png")

# %% [markdown]
# ### Visual 7 – Pair plot (multivariate view)

# %%
pp = sns.pairplot(df, vars=["mean_radius", "mean_texture", "mean_concave_points", "mean_smoothness",
"worst_area"],
                  hue="diagnosis", palette=PALETTE, corner=True, plot_kws=dict(alpha=0.55, s=18),
diag_kind="kde")
pp.figure.suptitle("Pair plot of five discriminative features", y=1.02, fontweight="bold")
pp.figure.savefig(FIG / "fig07_pairplot.png", dpi=130, bbox_inches="tight")
plt.show() if IN_NB else plt.close("all")

# %% [markdown]
# ### Visual 8 – Distribution fitting: histogram + Normal/Log-normal PDF and Q-Q plots (continuous
distributions)

# %%
fig, axes = plt.subplots(2, 3, figsize=(15, 8))
for r_i, f in enumerate(["mean_area", "mean_texture"]):
    x = df[f].values
    mu, sd = stats.norm.fit(x); sh, lc, sc = stats.lognorm.fit(x, floc=0)
    g = np.linspace(x.min(), x.max(), 300)
    axes[r_i, 0].hist(x, bins=30, density=True, color="#BBBBBB", edgecolor="white")
    axes[r_i, 0].plot(g, stats.norm.pdf(g, mu, sd), color="#0072B2", lw=2, label="Normal fit")
    axes[r_i, 0].plot(g, stats.lognorm.pdf(g, sh, lc, sc), color="#D55E00", lw=2, label="Log-normal fit")
    axes[r_i, 0].set(title=f"{f.replace('_',' ').title()} – fitted PDFs", xlabel=f); axes[r_i, 0].legend()
    stats.probplot(x, dist="norm", plot=axes[r_i, 1]); axes[r_i, 1].set_title("Q-Q plot vs Normal")
    stats.probplot(np.log(x), dist="norm", plot=axes[r_i, 2]); axes[r_i, 2].set_title("Q-Q plot of log(x)
vs Normal")
finish("fig08_distribution_fitting.png")

# %% [markdown]
# ### Visual 9 – Discrete distribution: Binomial model of malignant cases in a random sample

# %%
fig, ax = plt.subplots(figsize=(8, 4.5))
ax.hist(sim, bins=np.arange(-0.5, n_trial + 1.5, 1), density=True, color="#BBBBBB", edgecolor="white",
label="Monte-Carlo simulation (20,000 draws)")
ax.plot(k, pmf, "o-", color="#D55E00", lw=2, label=f"Binomial PMF  (n={n_trial}, p={p_mal:.3f})")
ax.axvline(n_trial * p_mal, color="k", ls="--", lw=1, label=f"Expected value = {n_trial*p_mal:.1f}")
ax.set(title="Number of malignant cases among 20 randomly chosen patients", xlabel="Malignant cases (k)",
ylabel="P(X = k)")
ax.legend()
finish("fig09_binomial_distribution.png")

# %% [markdown]
# ### Visual 10 – PCA: scree plot and 2-D projection

# %%
Xs = StandardScaler().fit_transform(df[FEATURES])
pcs = PCA(n_components=2).fit_transform(Xs)
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].bar(range(1, 11), pca_full.explained_variance_ratio_[:10] * 100, color="#56B4E9",
label="Individual")
axes[0].plot(range(1, 11), cum[:10] * 100, "o-", color="#D55E00", label="Cumulative")
axes[0].axhline(90, color="grey", ls="--", lw=1)
axes[0].set(title="Scree plot", xlabel="Principal component", ylabel="Explained variance (%)");
axes[0].legend()
sns.scatterplot(x=pcs[:, 0], y=pcs[:, 1], hue=df.diagnosis, palette=PALETTE, alpha=0.7, s=30, ax=axes[1])
axes[1].set(title="Samples in PC1 – PC2 space", xlabel=f"PC1 ({metrics['pca']['pc1']:.1%})", ylabel=f"PC2
({metrics['pca']['pc2']:.1%})")
finish("fig10_pca.png")

# %% [markdown]
# ## STAGE 6 – REFINE (transformation, encoding, scaling, splitting)
```

```
# * **Encode** target: `M → 1`, `B → 0` (malignant is the positive class).
# * **Split first** (stratified 70 % train / 15 % validation / 15 % test) so that nothing learned from
validation/test leaks into preprocessing.
# * **Transform** the 22 highly skewed (|skew| > 1) features with a **Yeo-Johnson power transform** – a
plain `log1p` was tried first
#   but barely changes features whose values are ≪ 1 (e.g. `concavity_error`), so it is not suitable here.
# * **Standardise** (z-score). Both the power transform and the scaler are **fitted on the training set
only**.

# %%
y = (df["diagnosis"] == "M").astype(int).values
X = df[FEATURES].copy()
skewed_cols = skew[skew.abs() > 1].index.tolist()

# 1) stratified split of the RAW features
X_tr, X_tmp, y_tr, y_tmp = train_test_split(X, y, test_size=0.30, stratify=y, random_state=SEED)
X_va, X_te, y_va, y_te = train_test_split(X_tmp, y_tmp, test_size=0.50, stratify=y_tmp, random_state=SEED)

# 2) fit the transformers on TRAIN only, then apply to train / validation / test
pt = PowerTransformer(method="yeo-johnson", standardize=False).fit(X_tr[skewed_cols])
def yj(Xdf):
    """Apply the (train-fitted) Yeo-Johnson transform to the skewed columns."""
    out = Xdf.copy(); out[skewed_cols] = pt.transform(Xdf[skewed_cols]); return out
scaler = StandardScaler().fit(yj(X_tr))
Z_tr, Z_va, Z_te = (pd.DataFrame(scaler.transform(yj(a)), columns=FEATURES) for a in (X_tr, X_va, X_te))

skew_after = yj(X).skew()
metrics["skew_mean_abs_before"] = round(float(skew.abs().mean()), 3)
metrics["skew_mean_abs_after"] = round(float(skew_after.abs().mean()), 3)
metrics["n_transformed"] = len(skewed_cols)
metrics["n_still_skewed"] = int((skew_after.abs() > 1).sum())
metrics["skew_examples"] = {c: [round(float(skew[c]), 2), round(float(skew_after[c]), 2)]
                            for c in ["area_error", "concavity_error", "fractal_dimension_error",
"perimeter_error", "mean_area"]}
print(f"Mean |skewness|: {metrics['skew_mean_abs_before']} -> {metrics['skew_mean_abs_after']} "
      f"(Yeo-Johnson on {len(skewed_cols)} features; still skewed: {metrics['n_still_skewed']})")
metrics["split"] = dict(train=len(y_tr), val=len(y_va), test=len(y_te),
                        malignant_train=int(y_tr.sum()), malignant_val=int(y_va.sum()),
malignant_test=int(y_te.sum()))
print(metrics["split"])
for name, Z, yy in (("train", Z_tr, y_tr), ("val", Z_va, y_va), ("test", Z_te, y_te)):
    out = Z.copy(); out["diagnosis"] = yy
    out.to_csv(PROC / f"{name}.csv", index=False)           # saved for reproducibility

# %% [markdown]
# ### Visual 11 – Effect of the power transform on skewness

# %%
fig, ax = plt.subplots(figsize=(7.5, 6.5))
lab = skew.abs().sort_values(ascending=False).head(12).index
ypos = np.arange(len(lab))
ax.barh(ypos + 0.2, skew[lab].abs().values, height=0.4, color="#D55E00", label="Before (raw)")
ax.barh(ypos - 0.2, skew_after[lab].abs().values, height=0.4, color="#0072B2", label="After Yeo-Johnson")
ax.set_yticks(ypos); ax.set_yticklabels([l.replace("_", " ") for l in lab]); ax.invert_yaxis()
ax.axvline(1, color="grey", ls="--", lw=1); ax.legend()
ax.set(title="Absolute skewness before vs after Yeo-Johnson (12 most skewed features)", xlabel="|
skewness|")
finish("fig11_skewness_before_after.png")

# %% [markdown]
# ## STAGE 7 – INTERACT (self-contained interactive explorer)
# Creates `interactive/scatter_explorer.html` – open it in any browser: choose any two features, toggle
classes, hover to inspect a patient.

# %%
payload = {"cols": FEATURES, "diag": df["diagnosis"].tolist(),
           "rows": df[FEATURES].round(4).values.tolist()}
html = r"""... HTML / CSS / JavaScript template of the interactive explorer (≈55 lines) – see
src/eda_mlp_pipeline.py in the repository ..."""
```

```
(INTER / "scatter_explorer.html").write_text(html.replace("__DATA__", json.dumps(payload, separators=(",",
":"))), encoding="utf-8")
print("Interactive explorer written to", (INTER / "scatter_explorer.html").relative_to(ROOT))

# %% [markdown]
# ## FEATURE ANALYSIS & FEATURE IMPORTANCE
# Three complementary views: Random-Forest (impurity) importance, Mutual Information, and permutation
importance of the
# final MLP (the last one is computed after the model is trained – see below).

# %%
rf = RandomForestClassifier(n_estimators=300, random_state=SEED).fit(Z_tr, y_tr)
imp_rf = pd.Series(rf.feature_importances_, index=FEATURES)
imp_mi = pd.Series(mutual_info_classif(Z_tr, y_tr, random_state=SEED), index=FEATURES)
print("Top-5 RF importance :", imp_rf.sort_values(ascending=False).head(5).round(3).to_dict())
print("Top-5 mutual info   :", imp_mi.sort_values(ascending=False).head(5).round(3).to_dict())

# %% [markdown]
# ## DEEP-LEARNING MODEL – Multi-Layer Perceptron (feed-forward neural network)
# * Input: 30 standardised features → ≥ 2 hidden layers (ReLU) → 1 sigmoid output (P(malignant)).
# * Optimiser: **Adam**, loss: binary cross-entropy (log-loss), mini-batch size 32, L2 regularisation
(`alpha`).
# * Training is done **epoch by epoch** so that loss/accuracy curves on train *and* validation sets can be
recorded.
# * **Early stopping** on validation loss (patience = 20) – the weights of the best epoch are restored.

# %%
def train_mlp(hidden, alpha, lr, Xtr, ytr, Xva, yva, max_epochs=200, patience=20, seed=SEED):
    """Train an MLP epoch-by-epoch; return (best_model, history, best_epoch)."""
    clf = MLPClassifier(hidden_layer_sizes=hidden, activation="relu", solver="adam", alpha=alpha,
                        learning_rate_init=lr, batch_size=32, random_state=seed)
    hist = {"loss": [], "val_loss": [], "acc": [], "val_acc": []}
    best_loss, best_clf, best_ep, wait = np.inf, None, 0, 0
    for ep in range(1, max_epochs + 1):
        clf.partial_fit(Xtr, ytr, classes=[0, 1])            # one pass over the training data = one epoch
        p_tr, p_va = clf.predict_proba(Xtr)[:, 1], clf.predict_proba(Xva)[:, 1]
        hist["loss"].append(log_loss(ytr, p_tr)); hist["val_loss"].append(log_loss(yva, p_va))
        hist["acc"].append(accuracy_score(ytr, p_tr > .5)); hist["val_acc"].append(accuracy_score(yva, p_va
> .5))
        if hist["val_loss"][-1] < best_loss - 1e-5:
            best_loss, best_clf, best_ep, wait = hist["val_loss"][-1], copy.deepcopy(clf), ep, 0
        else:
            wait += 1
            if wait >= patience:
                break
    return best_clf, hist, best_ep

# %% [markdown]
# ### Basic hyper-parameter tuning (grid search on the **validation** set)
# Grid: hidden-layer architecture (all ≥ 2 hidden layers, i.e. a *deep* network) × L2 penalty (`alpha`) ×
learning rate
# = 4 × 2 × 3 = 24 configurations. Selection criterion: highest validation accuracy, ties broken by lowest
validation loss.

# %%
LRS = [2.5e-4, 5e-4, 1e-3]      # lr >= 5e-3 was tried first: validation loss bottomed out after only ~2
epochs (unstable), so the grid is centred on gentler rates
HIDDEN = [(32, 16), (64, 32), (64, 32, 16), (128, 64, 32)]
grid = [(h, a, lr) for h in HIDDEN for a in [1e-4, 1e-2] for lr in LRS]
tune = []
for h, a, lr in grid:
    mdl, hs, be = train_mlp(h, a, lr, Z_tr, y_tr, Z_va, y_va)
    pv = mdl.predict_proba(Z_va)[:, 1]
    tune.append(dict(hidden=str(h), alpha=a, lr=lr, best_epoch=be, val_loss=log_loss(y_va, pv),
                     val_acc=accuracy_score(y_va, pv > .5), val_auc=roc_auc_score(y_va, pv)))
tune_df = pd.DataFrame(tune).sort_values(["val_acc", "val_loss"], ascending=[False,
True]).reset_index(drop=True)
tune_df.to_csv(OUT / "hyperparameter_tuning_results.csv", index=False)
display(tune_df.head(8).round(4))
best = tune_df.iloc[0]
```

```
BEST = dict(hidden=eval(best.hidden), alpha=float(best.alpha), lr=float(best.lr))
print("\nBest configuration:", BEST)
metrics["tuning"] = dict(n_configs=len(grid), best=dict(hidden=best.hidden, alpha=float(best.alpha),
lr=float(best.lr),
                         best_epoch=int(best.best_epoch), val_acc=round(float(best.val_acc), 4),
val_loss=round(float(best.val_loss), 4)),
                         worst_val_acc=round(float(tune_df.val_acc.min()), 4),
top_table=tune_df.head(5).round(4).to_dict("records"))

# %% [markdown]
# ### Visual 12 – Hyper-parameter tuning heat-map

# %%
fig, axes = plt.subplots(1, len(LRS), figsize=(5.2 * len(LRS), 4.2))
for i_, (ax, lr) in enumerate(zip(axes, LRS)):
    pv_ = tune_df[tune_df.lr == lr].pivot(index="hidden", columns="alpha",
values="val_acc").reindex([str(h) for h in HIDDEN])
    sns.heatmap(pv_, annot=True, fmt=".3f", cmap="YlGnBu", vmin=tune_df.val_acc.min() - .01, vmax=1,
cbar=False, ax=ax)
    ax.set(title=f"Validation accuracy – learning rate {lr}", xlabel="L2 penalty (alpha)", ylabel="Hidden
layers" if i_ == 0 else "")
    ax.tick_params(axis="y", rotation=0)
    if i_ > 0:
        ax.set_yticklabels([])
finish("fig12_hyperparameter_tuning.png")

# %% [markdown]
# ### Train the final model with the best configuration

# %%
mlp, hist, best_ep = train_mlp(BEST["hidden"], BEST["alpha"], BEST["lr"], Z_tr, y_tr, Z_va, y_va)
n_params = sum(w.size for w in mlp.coefs_) + sum(b.size for b in mlp.intercepts_)
print(f"Architecture: 30 -> {' -> '.join(map(str, BEST['hidden']))} -> 1 | trainable parameters =
{n_params:,}")
print(f"Epochs run = {len(hist['loss'])}, best epoch (restored) = {best_ep}")
metrics["model"] = dict(hidden=str(BEST["hidden"]), n_params=int(n_params), epochs_run=len(hist["loss"]),
best_epoch=int(best_ep),
                        alpha=BEST["alpha"], lr=BEST["lr"], batch_size=32, optimizer="Adam",
activation="ReLU (hidden) / sigmoid (output)",
                        patience=20, max_epochs=200)

# %% [markdown]
# ## MODEL EVALUATION VISUALISATIONS
# ### Visual 13 – Loss vs Epoch and Accuracy vs Epoch

# %%
ep = np.arange(1, len(hist["loss"]) + 1)
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
axes[0].plot(ep, hist["loss"], color="#0072B2", lw=2, label="Training loss")
axes[0].plot(ep, hist["val_loss"], color="#D55E00", lw=2, label="Validation loss")
axes[0].axvline(best_ep, color="grey", ls="--", label=f"Best epoch = {best_ep}")
axes[0].set(title="Loss vs Epoch", xlabel="Epoch", ylabel="Binary cross-entropy"); axes[0].legend()
axes[1].plot(ep, hist["acc"], color="#0072B2", lw=2, label="Training accuracy")
axes[1].plot(ep, hist["val_acc"], color="#D55E00", lw=2, label="Validation accuracy")
axes[1].axvline(best_ep, color="grey", ls="--")
axes[1].set(title="Accuracy vs Epoch", xlabel="Epoch", ylabel="Accuracy", ylim=(0.55, 1.01));
axes[1].legend(loc="lower right")
finish("fig13_loss_accuracy_curves.png")

# %%
# ---- Test-set evaluation (the test set is used ONCE, only now) ----
def evaluate(name, proba, y_true, thr=0.5):
    pred = (proba >= thr).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_true, pred).ravel()
    return dict(model=name, accuracy=accuracy_score(y_true, pred), precision=precision_score(y_true, pred),
                recall=recall_score(y_true, pred), specificity=tn / (tn + fp), f1=f1_score(y_true, pred),
                auc=roc_auc_score(y_true, proba), avg_precision=average_precision_score(y_true, proba),
                log_loss=log_loss(y_true, proba), tn=int(tn), fp=int(fp), fn=int(fn), tp=int(tp))

p_mlp = mlp.predict_proba(Z_te)[:, 1]
```

```
lr_base = LogisticRegression(max_iter=2000, random_state=SEED).fit(Z_tr, y_tr)
p_lr, p_rf = lr_base.predict_proba(Z_te)[:, 1], rf.predict_proba(Z_te)[:, 1]
results = pd.DataFrame([evaluate("MLP (deep NN)", p_mlp, y_te), evaluate("Logistic Regression", p_lr,
y_te),
                        evaluate("Random Forest", p_rf, y_te)])
results.to_csv(OUT / "test_set_results.csv", index=False)
display(results.round(4))
metrics["test_results"] = results.round(4).to_dict("records")

# 5-fold stratified CV on train+val: the power transform + scaler sit INSIDE the pipeline (re-fitted per
fold -> no leakage)
X_trva, y_trva = pd.concat([X_tr, X_va]), np.concatenate([y_tr, y_va])
skew_idx = [FEATURES.index(c) for c in skewed_cols]
cv_pipe = make_pipeline(ColumnTransformer([("yj", PowerTransformer(standardize=False), skew_idx)],
remainder="passthrough"),
                        StandardScaler(),
                        MLPClassifier(hidden_layer_sizes=BEST["hidden"], alpha=BEST["alpha"],
learning_rate_init=BEST["lr"],
                                      batch_size=32, max_iter=max(best_ep, 30), random_state=SEED))
cv = StratifiedKFold(5, shuffle=True, random_state=SEED)
cv_acc = cross_val_score(cv_pipe, X_trva, y_trva, cv=cv, scoring="accuracy")
cv_auc = cross_val_score(cv_pipe, X_trva, y_trva, cv=cv, scoring="roc_auc")
metrics["cv"] = dict(acc_mean=round(float(cv_acc.mean()), 4), acc_std=round(float(cv_acc.std()), 4),
                     auc_mean=round(float(cv_auc.mean()), 4), auc_std=round(float(cv_auc.std()), 4))
print("5-fold CV  accuracy = %.4f ± %.4f | AUC = %.4f ± %.4f" % (cv_acc.mean(), cv_acc.std(),
cv_auc.mean(), cv_auc.std()))

# %% [markdown]
# ### Visual 14 – Confusion matrix (test set)

# %%
fig, axes = plt.subplots(1, 2, figsize=(11, 4.6))
cm = confusion_matrix(y_te, (p_mlp >= .5).astype(int))
ConfusionMatrixDisplay(cm, display_labels=["Benign", "Malignant"]).plot(ax=axes[0], cmap="Blues",
colorbar=False, values_format="d")
axes[0].set_title("Confusion matrix (counts)"); axes[0].grid(False)
ConfusionMatrixDisplay(cm / cm.sum(1, keepdims=True), display_labels=["Benign",
"Malignant"]).plot(ax=axes[1], cmap="Blues", colorbar=False, values_format=".2%")
axes[1].set_title("Confusion matrix (row-normalised)"); axes[1].grid(False)
finish("fig14_confusion_matrix.png")

# %% [markdown]
# ### Visual 15 – ROC curve with AUC, and Precision–Recall curve

# %%
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
for name, p_, c in (("MLP", p_mlp, "#D55E00"), ("Logistic Regression", p_lr, "#0072B2"), ("Random Forest",
p_rf, "#009E73")):
    fpr, tpr, _ = roc_curve(y_te, p_)
    axes[0].plot(fpr, tpr, lw=2.2 if name == "MLP" else 1.6, color=c, label=f"{name} (AUC =
{roc_auc_score(y_te, p_):.3f})")
    pr, rc, _ = precision_recall_curve(y_te, p_)
    axes[1].plot(rc, pr, lw=2.2 if name == "MLP" else 1.6, color=c, label=f"{name} (AP =
{average_precision_score(y_te, p_):.3f})")
axes[0].plot([0, 1], [0, 1], "k--", lw=1, label="Chance")
axes[0].set(title="ROC curve (test set)", xlabel="False positive rate (1 – specificity)", ylabel="True
positive rate (recall)"); axes[0].legend(loc="lower right")
axes[1].set(title="Precision–Recall curve (test set)", xlabel="Recall", ylabel="Precision", ylim=(0.5,
1.02)); axes[1].legend(loc="lower left")
finish("fig15_roc_pr_curves.png")

# %% [markdown]
# ### Visual 16 – Error analysis: predicted-probability distribution and residuals

# %%
resid = y_te - p_mlp                        # signed error (true label minus predicted probability)
wrong = (p_mlp >= .5).astype(int) != y_te
fig, axes = plt.subplots(1, 2, figsize=(13, 4.8))
axes[0].hist(p_mlp[y_te == 0], bins=20, range=(0, 1), alpha=.7, color=PALETTE["B"], label="Actual Benign")
```

```
axes[0].hist(p_mlp[y_te == 1], bins=20, range=(0, 1), alpha=.7, color=PALETTE["M"], label="Actual
Malignant")
axes[0].axvline(.5, color="k", ls="--", lw=1, label="Decision threshold 0.5")
axes[0].set(title="Predicted P(malignant) by true class", xlabel="Predicted probability",
ylabel="Patients"); axes[0].legend()
sns.histplot(resid, bins=25, kde=True, color="#7f7f7f", ax=axes[1])
axes[1].axvline(0, color="k", lw=1)
axes[1].set(title=f"Prediction-error (residual) distribution – {int(wrong.sum())} misclassified of
{len(y_te)}",
            xlabel="Residual = true label – predicted probability")
finish("fig16_error_distribution.png")
metrics["error"] = dict(n_wrong=int(wrong.sum()), resid_mean=round(float(resid.mean()), 4),
resid_sd=round(float(resid.std()), 4),
                        wrong_probs=[round(float(v), 3) for v in p_mlp[wrong]], wrong_truth=[int(v) for v
in y_te[wrong]],
                        share_confident=round(float(((p_mlp < .05) | (p_mlp > .95)).mean()), 3))

# %% [markdown]
# ### Visual 17 – Feature importance (RF, mutual information, permutation importance of the MLP)

# %%
perm = permutation_importance(mlp, Z_va, y_va, scoring="neg_log_loss", n_repeats=30, random_state=SEED)
imp_perm = pd.Series(perm.importances_mean, index=FEATURES)
fig, axes = plt.subplots(1, 3, figsize=(17, 5.2))
for ax, s_, ttl, col in ((axes[0], imp_rf, "Random-Forest importance", "#009E73"), (axes[1], imp_mi,
"Mutual information", "#56B4E9"),
                         (axes[2], imp_perm, "MLP permutation importance (rise in val. log-loss)",
"#D55E00")):
    t = s_.sort_values(ascending=False).head(10)[::-1]
    ax.barh([i.replace("_", " ") for i in t.index], t.values, color=col); ax.set_title(ttl, fontsize=11)
finish("fig17_feature_importance.png")
metrics["importance"] = dict(rf=imp_rf.sort_values(ascending=False).head(5).round(3).to_dict(),
                             mi=imp_mi.sort_values(ascending=False).head(5).round(3).to_dict(),
                             perm=imp_perm.sort_values(ascending=False).head(5).round(4).to_dict())
# Rule of three: with 0 errors in n test cases, the 95 % upper bound of the true error rate is about 3/n
metrics["rule_of_three_err_upper"] = round(3 / len(y_te), 4)
print(metrics["importance"])

# %% [markdown]
# ## SAVE ALL NUMBERS USED IN THE REPORT

# %%
metrics["n_figures"] = len(list(FIG.glob("fig*.png")))
with open(OUT / "metrics.json", "w") as fh:
    json.dump(metrics, fh, indent=2, default=lambda o: o.item() if hasattr(o, "item") else str(o))
print("Saved outputs/metrics.json |", metrics["n_figures"], "figures in figures/")
```

Appendix B – GitHub Repository Structure

Public repository: https://github.com/your-username/breast-cancer-eda-visualization

```
breast-cancer-eda-visualization/
├── README.md                      project description, results, how to run
├── requirements.txt               Python dependencies
├── .gitignore
├── data/
│   ├── breast_cancer_wisconsin.csv    raw dataset (UCI WDBC)
│   ├── DATA_DICTIONARY.md             description of every field + source link
│   └── processed/                     train.csv, val.csv, test.csv
├── notebooks/
│   └── EDA_Visualization_MLP.ipynb    executed notebook (outputs included)
├── src/
│   └── eda_mlp_pipeline.py            runnable script (same code)
├── figures/                       17 figures + interactive-explorer screenshot
├── interactive/
│   └── scatter_explorer.html          interactive explorer (stage 7)
├── outputs/                       metrics.json and result tables (CSV)
└── reports/
    ├── Project_Report.docx            this report (Word)
    └── Project_Report.pdf             this report (PDF)
```